

ARTIFICIAL INTELLIGENCE

Lab Handbook

Beginner's Edition — Learn It From Scratch

No prior programming experience assumed.

Every line of code is explained. Every command is written out.

Course Code: [Insert Course Code]
Instructor: [Insert Instructor Name]
Semester: [Insert Semester / Year]

Contents

How To Use This Handbook	3
Lab 1: Introduction to Python for AI — Setup and Basic Data Handling	4
Lab 2: Implement a Simple Rational Agent in a Defined Environment	9
Lab 3: Implement & Visualize BFS/DFS for a Maze Pathfinding Problem	14
Lab 4: Compare Performance of Uninformed vs. Informed Search	21
Lab 5: Implement A* Search to Solve the 8-Puzzle Problem	28
Lab 6: Implement Hill Climbing; Analyze Its Tendency to Get Stuck	35
Lab 7: Develop a Tic-Tac-Toe Game with the Min-Max Algorithm	42
Lab 8: Enhance the Game with Alpha-Beta Pruning and Compare Efficiency	49
Lab 9: Build a Small Rule-Based Expert System	55
Lab 10: Build a Decision Tree Classifier From Scratch	60
Lab 11: Implement a Naïve Bayes Classifier for Spam Detection	67
Lab 12: Compare Naïve Bayes and SVM Using scikit-learn	73
Lab 13: Build and Train a Feedforward Neural Network for Digit Classification	79
Lab 14: Implement an MLP From Scratch to Solve XOR	86
Lab 15: Implement a Genetic Algorithm to Solve Eight Queens	93
Lab 16: Build a Small AI Application Using a Pre-Trained CNN	100

How To Use This Handbook

This edition assumes you have never written a line of Python. Every lab is broken into small steps, and every step shows the code plus a table explaining what each line does. You are meant to type the code yourself rather than copy it — typing is how the syntax becomes familiar.

What each lab contains

- What this lab is about — the idea in plain English, before any code.
- Before you start — what must already be installed or understood.
- Building it step by step — small pieces of code, each explained line by line.
- The complete program — everything assembled into one runnable file.
- How to run it — the exact commands to type.
- What you should see — the output, so you can confirm it worked.
- If something goes wrong — the errors beginners actually hit, and their fixes.
- Try it yourself — small modifications that deepen understanding.

One-time setup

Install Python 3.10 or newer from python.org. On Windows, tick 'Add Python to PATH' during installation. Then open a terminal and run:

```
python --version
pip install numpy pandas matplotlib scikit-learn
```

Labs 13 and 16 additionally need TensorFlow. It is a large download, so install it only when you reach those labs:

```
pip install tensorflow pillow
```

A note on typing versus copying

Copying code gets a program running. Typing it gets the language into your fingers. You will make typos, see error messages, and fix them — and that process is where most of the actual learning happens. The troubleshooting table in every lab exists because those mistakes are expected, not shameful.

Working folder

Create one folder for all your work, for example `ai_labs`, and save each lab as `lab1.py`, `lab2.py` and so on. Run every program from inside that folder.

```
mkdir ai_labs
cd ai_labs
```

LAB 1: Introduction to Python for AI — Setup and Basic Data Handling

Relates to: Week 1 — Introduction to AI, Intelligent Agents

WHAT THIS LAB IS ABOUT

Before you can build anything intelligent, you need a way to store information and do arithmetic on it quickly. That is all this lab is. Python gives you four tools you will use in every single remaining lab: a list (an ordered box of items), a dictionary (a labelled box, where each value has a name), a NumPy array (a list of numbers that supports fast maths on the whole list at once), and a Pandas DataFrame (a spreadsheet in code, with named columns and rows). If you understand these four, every later lab becomes readable.

BEFORE YOU START, YOU NEED

- Python 3.10 or newer installed on your machine
- A text editor — VS Code, PyCharm, IDLE, or even Notepad will work
- A terminal (Command Prompt on Windows, Terminal on Mac/Linux)
- No prior Python experience assumed

Building it step by step

Step 1 — Install Python and check it works

Download Python from python.org and install it. On Windows, tick the box labelled 'Add Python to PATH' during installation — this is the single most common setup mistake. Then open a terminal and check the installation responded.

```
python --version

# You should see something like: Python 3.12.1
# On Mac/Linux you may need to type python3 instead of python
```

Step 2 — Install the libraries you will use all semester

Python on its own cannot do fast maths or read spreadsheets. These four packages add those abilities. You only need to run this command once.

```
pip install numpy pandas matplotlib scikit-learn

# On Mac/Linux, if pip is not found, try:
# pip3 install numpy pandas matplotlib scikit-learn
```

Line by line:

Code	What it does
numpy	Fast numerical arrays and maths
pandas	Tables of data with named columns
matplotlib	Drawing charts and graphs
scikit-learn	Ready-made machine learning algorithms (used from Lab 12 onward)

Step 3 — Create your first file and store some data

Make a folder called `ai_labs`, and inside it create a file called `lab1.py`. Type the following. A dictionary uses curly braces, and each entry is a name, then a colon, then a value. We are describing the state of an imaginary robot.

```
agent_state = {
    'position': (2, 3),
    'energy': 87,
    'sensors': ['bump', 'clear', 'clear', 'bump']
```

```

}

print('Agent state:', agent_state)
print('Energy level:', agent_state['energy'])

```

Line by line:

Code	What it does
<code>agent_state = { ... }</code>	Creates a dictionary: a set of named values
<code>'position': (2, 3)</code>	A tuple — a fixed pair of numbers, here an x/y coordinate
<code>'sensors': [...]</code>	A list — an ordered, changeable collection of items
<code>agent_state['energy']</code>	Reads one value back out by its name

Step 4 — Do fast maths with NumPy

A normal Python list cannot be divided or averaged directly. A NumPy array can. Notice that `readings.mean()` computes the average of all five numbers in one instruction, with no loop.

```

import numpy as np

readings = np.array([12.5, 9.3, 15.7, 20.1, 8.4])

print('All readings:', readings)
print('Average:', readings.mean())
print('Highest:', readings.max())
print('Lowest:', readings.min())

# Normalising: rescale every number to sit between 0 and 1
normalized = (readings - readings.min()) / (readings.max() - readings.min())
print('Normalized:', normalized)

```

Line by line:

Code	What it does
<code>import numpy as np</code>	Loads NumPy and gives it the short nickname np
<code>np.array([...])</code>	Turns a plain Python list into a NumPy array
<code>readings.mean()</code>	Average of every number in the array
<code>readings - readings.min()</code>	Subtracts one number from EVERY element at once — no loop needed
<code>normalized</code>	Every value now sits between 0.0 and 1.0; used constantly before feeding data to neural networks

Step 5 — Handle a table of data with Pandas

A `DataFrame` is a spreadsheet in code. Real datasets always have missing values, so we practise filling one in, then selecting only the rows we care about.

```

import pandas as pd

df = pd.DataFrame({
    'trial':    [1, 2, 3, 4, 5],
    'success':  [1, 0, 1, 1, None],
    'time_sec': [3.2, 5.1, 2.8, 4.0, 3.9]
})

print(df)

```

```

print()
print('Summary statistics:')
print(df.describe())

# Fill the missing value in the success column with 0
df['success'] = df['success'].fillna(0)

# Keep only rows where time_sec is under 4 seconds
fast_trials = df[df['time_sec'] < 4.0]
print()
print('Fast trials only:')
print(fast_trials)

```

Line by line:

Code	What it does
<code>pd.DataFrame({...})</code>	Builds a table; each dictionary key becomes a column name
<code>None</code>	Python's way of saying 'this value is missing'
<code>df.describe()</code>	Prints count, mean, min, max and quartiles for every numeric column
<code>df['success'].fillna(0)</code>	Replaces every missing value in that column with 0
<code>df[df['time_sec'] < 4.0]</code>	Keeps only the rows where the condition is true — this is called boolean filtering

Step 6 — Draw your first chart

We save the chart to a file rather than showing it on screen, because saving works reliably everywhere, including on servers with no display attached.

```

import matplotlib.pyplot as plt

plt.hist(readings, bins=5)
plt.title('Sensor Reading Distribution')
plt.xlabel('Reading value')
plt.ylabel('How many times it occurred')
plt.savefig('lab1_histogram.png')
print('Chart saved as lab1_histogram.png')

```

Line by line:

Code	What it does
<code>plt.hist(readings, bins=5)</code>	Draws a histogram, splitting the values into 5 buckets
<code>plt.savefig('...')</code>	Writes the image to a file in your current folder

The complete program (lab1.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab1.py – Introduction to Python for AI
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# --- Part 1: dictionaries, tuples and lists ---
agent_state = {
    'position': (2, 3),
    'energy': 87,
    'sensors': ['bump', 'clear', 'clear', 'bump']
}
print('Agent state:', agent_state)
print('Energy level:', agent_state['energy'])
print()

# --- Part 2: NumPy arrays ---
readings = np.array([12.5, 9.3, 15.7, 20.1, 8.4])
print('Average:', readings.mean())
print('Standard deviation:', readings.std())
normalized = (readings - readings.min()) / (readings.max() - readings.min())
print('Normalized:', normalized)
print()

# --- Part 3: Pandas DataFrames ---
df = pd.DataFrame({
    'trial':    [1, 2, 3, 4, 5],
    'success':  [1, 0, 1, 1, None],
    'time_sec': [3.2, 5.1, 2.8, 4.0, 3.9]
})
print(df.describe())
df['success'] = df['success'].fillna(0)
fast_trials = df[df['time_sec'] < 4.0]
print()
print('Fast trials:')
print(fast_trials)

# --- Part 4: a chart ---
plt.hist(readings, bins=5)
plt.title('Sensor Reading Distribution')
plt.xlabel('Reading value')
plt.ylabel('How many times it occurred')
plt.savefig('lab1_histogram.png')
print()
print('Chart saved as lab1_histogram.png')
```

HOW TO RUN IT

```
cd path/to/ai_labs
python lab1.py
```

WHAT YOU SHOULD SEE

```
Agent state: {'position': (2, 3), 'energy': 87, 'sensors': [...]}
Energy level: 87

Average: 13.2
Standard deviation: 4.19...
```

```
Normalized: [0.349 0.077 0.622 1.    0.    ]
```

```

      trial  success  time_sec
count    5.0  4.000000  5.000000
mean     3.0  0.750000  3.800000
...
```

```
Fast trials:
```

```

      trial  success  time_sec
0         1        1.0        3.2
2         3        1.0        2.8
4         5        0.0        3.9
```

```
Chart saved as lab1_histogram.png
```

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
'python' is not recognized	Python was not added to your system PATH during installation	Reinstall Python and tick 'Add Python to PATH', or type the full path to python.exe
ModuleNotFoundError: No module named 'numpy'	The libraries were never installed, or installed for a different Python version	Run: <code>pip install numpy pandas matplotlib scikit-learn</code>
IndentationError	Python uses indentation to group code; spaces and tabs were mixed	Use 4 spaces consistently; never mix tabs and spaces
FileNotFoundError when saving the chart	You ran the script from a folder you do not have permission to write to	<code>cd</code> into your own project folder before running

TRY IT YOURSELF

- Add a fourth key to `agent_state` called 'name' and print it.
- Compute the median of readings (hint: `np.median(readings)`).
- Filter the DataFrame to show only the rows where success equals 1.
- Change the histogram to 3 bins instead of 5 and observe how the shape changes.

LAB 2: Implement a Simple Rational Agent in a Defined Environment

Relates to: Week 1 — Intelligent Agents, PEAS, Agent Types

WHAT THIS LAB IS ABOUT

An agent is anything that senses its world and acts on it. A thermostat is an agent: it senses temperature and switches a heater. In this lab you build a robot vacuum living in a world with exactly two rooms, A and B. Each room is either Clean or Dirty. The vacuum can Suck, move Left, or move Right. You will write two versions: a simple reflex agent that only reacts to what it can see right now, and a model-based agent that remembers what it has already cleaned. Then you measure which one performs better.

BEFORE YOU START, YOU NEED

- Completion of Lab 1
- Understanding of Python functions (def) and classes (class)
- No extra libraries — only Python's built-in random module

Building it step by step

Step 1 — Build the world

The environment is a class: a blueprint that bundles data (which rooms are dirty, where the vacuum is) together with the actions that change that data. `__init__` is the setup function that runs automatically when you create a new environment.

```
import random

class VacuumEnvironment:
    def __init__(self):
        # Randomly decide whether each room starts clean or dirty
        self.status = {
            'A': random.choice(['Clean', 'Dirty']),
            'B': random.choice(['Clean', 'Dirty'])
        }
        self.agent_location = random.choice(['A', 'B'])
        self.performance = 0
```

Line by line:

Code	What it does
<code>class VacuumEnvironment:</code>	Defines a new type of object
<code>def __init__(self):</code>	Runs automatically when the object is created; sets up its starting state
<code>self</code>	Refers to this particular object, so each environment keeps its own separate data
<code>random.choice(...)</code>	Picks one item from the list at random
<code>self.performance = 0</code>	The running score we will use to judge how well the agent did

Step 2 — Let the world be sensed and acted upon

`percept` returns what the agent can currently see: only its own location and whether that one room is dirty. It cannot see the other room — this limitation is the whole point. `execute` applies an action and updates the score.

```
def percept(self):
    # The agent only sees the room it is standing in
    return (self.agent_location, self.status[self.agent_location])

def execute(self, action):
    if action == 'Suck':
```

```

        if self.status[self.agent_location] == 'Dirty':
            self.status[self.agent_location] = 'Clean'
    elif action == 'Right':
        self.agent_location = 'B'
        self.performance -= 1      # moving costs 1 point
    elif action == 'Left':
        self.agent_location = 'A'
        self.performance -= 1

    # Award 1 point for every room that is currently clean
    for room in self.status:
        if self.status[room] == 'Clean':
            self.performance += 1

```

Line by line:

Code	What it does
<code>return (location, status)</code>	Returns a tuple — a pair of values bundled together
<code>elif</code>	Short for 'else if' — checked only if the previous conditions were false
<code>self.performance -= 1</code>	Subtracts 1; penalises pointless wandering
<code>for room in self.status:</code>	Loops over every room name in the dictionary

Step 3 — Write the simple reflex agent

This is the simplest possible agent: one rule. If the current room is dirty, clean it. Otherwise move to the other room. It has no memory whatsoever, so it will keep pacing back and forth forever even after both rooms are clean.

```

def simple_reflex_agent(percept):
    location, status = percept      # unpack the tuple into two variables

    if status == 'Dirty':
        return 'Suck'
    if location == 'A':
        return 'Right'
    return 'Left'

```

Line by line:

Code	What it does
<code>location, status = percept</code>	Tuple unpacking: splits the pair into two named variables
<code>return 'Suck'</code>	The function immediately stops and hands back this action
No memory anywhere	This is exactly what makes it a SIMPLE REFLEX agent

Step 4 — Write the model-based agent

This agent keeps an internal model: a dictionary recording what it has learned about each room. Once it knows both rooms are clean, it stops moving and returns NoOp (no operation), saving the movement penalty.

```

class ModelBasedAgent:
    def __init__(self):
        # Start out knowing nothing about either room
        self.model = {'A': None, 'B': None}

    def act(self, percept):
        location, status = percept
        self.model[location] = status      # remember what we just saw

```

```

    if status == 'Dirty':
        return 'Suck'

    # If we know BOTH rooms are clean, stop wasting moves
    if self.model['A'] == 'Clean' and self.model['B'] == 'Clean':
        return 'NoOp'

    return 'Right' if location == 'A' else 'Left'

```

Line by line:

Code	What it does
<code>self.model</code>	The internal memory — this is the ONLY difference from the reflex agent
<code>None</code>	Means 'I have not seen this room yet'
<code>'Right' if location == 'A' else 'Left'</code>	A one-line if/else expression, called a ternary
<code>return 'NoOp'</code>	Do nothing — avoids losing a point for a pointless move

Step 5 — Run a trial and compare

We use `random.seed(42)` before each run. A seed forces the random number generator to produce the same sequence every time, so both agents face an identical starting world. Without this, the comparison would be meaningless.

```

def run_trial(agent_type, steps=6):
    env = VacuumEnvironment()
    model_agent = ModelBasedAgent() if agent_type == 'model' else None

    for step in range(steps):
        percept = env.percept()
        if agent_type == 'reflex':
            action = simple_reflex_agent(percept)
        else:
            action = model_agent.act(percept)
        env.execute(action)

    return env.performance

random.seed(42)
reflex_score = run_trial('reflex')

random.seed(42)      # reset, so the model agent gets the SAME starting world
model_score = run_trial('model')

print('Simple Reflex Agent performance:', reflex_score)
print('Model-Based Agent performance: ', model_score)

```

Line by line:

Code	What it does
<code>random.seed(42)</code>	Makes randomness repeatable — essential for fair comparison
<code>for step in range(steps):</code>	Repeats the sense-think-act cycle a fixed number of times
<code>env.percept()</code>	SENSE
<code>simple_reflex_agent(percept)</code>	THINK — decide on an action
<code>env.execute(action)</code>	ACT — change the world

The complete program (lab2.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab2.py – A rational agent in a vacuum world
import random

class VacuumEnvironment:
    def __init__(self):
        self.status = {
            'A': random.choice(['Clean', 'Dirty']),
            'B': random.choice(['Clean', 'Dirty'])
        }
        self.agent_location = random.choice(['A', 'B'])
        self.performance = 0

    def percept(self):
        return (self.agent_location, self.status[self.agent_location])

    def execute(self, action):
        if action == 'Suck':
            if self.status[self.agent_location] == 'Dirty':
                self.status[self.agent_location] = 'Clean'
        elif action == 'Right':
            self.agent_location = 'B'
            self.performance -= 1
        elif action == 'Left':
            self.agent_location = 'A'
            self.performance -= 1
        for room in self.status:
            if self.status[room] == 'Clean':
                self.performance += 1

def simple_reflex_agent(percept):
    location, status = percept
    if status == 'Dirty':
        return 'Suck'
    if location == 'A':
        return 'Right'
    return 'Left'

class ModelBasedAgent:
    def __init__(self):
        self.model = {'A': None, 'B': None}

    def act(self, percept):
        location, status = percept
        self.model[location] = status
        if status == 'Dirty':
            return 'Suck'
        if self.model['A'] == 'Clean' and self.model['B'] == 'Clean':
            return 'NoOp'
        return 'Right' if location == 'A' else 'Left'

def run_trial(agent_type, steps=6):
    env = VacuumEnvironment()
    model_agent = ModelBasedAgent() if agent_type == 'model' else None
    for step in range(steps):
        percept = env.percept()
        if agent_type == 'reflex':
```

```

        action = simple_reflex_agent(percept)
    else:
        action = model_agent.act(percept)
    env.execute(action)
    return env.performance

random.seed(42)
reflex_score = run_trial('reflex')
random.seed(42)
model_score = run_trial('model')

print('Simple Reflex Agent performance:', reflex_score)
print('Model-Based Agent performance: ', model_score)

```

HOW TO RUN IT

```
python lab2.py
```

WHAT YOU SHOULD SEE

```
Simple Reflex Agent performance: 7
Model-Based Agent performance: 10
```

(Exact numbers depend on the random starting world, but the model-based agent should score the same or higher, because it stops moving once it knows both rooms are clean, avoiding the -1 movement penalty.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
NameError: name 'self' is not defined	A method was written outside the class, or indentation was lost	Every method must be indented 4 spaces inside the class block
TypeError: percept() takes 1 positional argument but 2 were given	Called env.percept(something) instead of env.percept()	percept takes no arguments other than self
Both agents score identically	You forgot to reset random.seed before the second run	Call random.seed(42) again before running the second trial
AttributeError: 'NoneType' object has no attribute 'act'	agent_type was not exactly the string 'model'	Check spelling of the agent_type argument

TRY IT YOURSELF

- Increase steps from 6 to 20. Does the performance gap widen? Why?
- Add a third room 'C' and adjust both agents to handle it.
- Make dirt reappear randomly each step, and see which agent copes better.
- Write a goal-based agent that plans the shortest route to clean both rooms.

LAB 3: Implement & Visualize BFS/DFS for a Maze Pathfinding Problem

Relates to: Week 3 — Uninformed Search: BFS, DFS, Iterative Deepening

WHAT THIS LAB IS ABOUT

A maze is just a set of connected points. Searching it means asking: starting here, which points can I reach, and in what order should I check them? Breadth-First Search checks everything one step away, then everything two steps away, and so on — like ripples spreading from a stone. It always finds the shortest path. Depth-First Search picks one direction and runs as far as it can before backing up — like following one corridor to its dead end. It uses far less memory but may return a needlessly long path. The only difference in code is one line: whether you take items from the front or the back of your to-do list.

BEFORE YOU START, YOU NEED

- Completion of Labs 1 and 2
- Understanding of Python lists, dictionaries, sets, and while loops
- matplotlib and numpy installed

Building it step by step

Step 1 — Represent the maze

We use a grid of numbers. 0 means you can walk there, 1 means it is a wall. Positions are written as (row, column), counting from 0 at the top-left.

```
maze = [
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)    # top-left corner
goal  = (4, 4)    # bottom-right corner
```

Line by line:

Code	What it does
<code>maze = [...], [...]</code>	A list of lists — each inner list is one row of the grid
<code>0</code>	An open, walkable cell
<code>1</code>	A wall you cannot pass through
<code>(0, 0)</code>	Row 0, column 0 — Python counts from zero

Step 2 — Find which cells you can move to

From any cell you may move up, down, left or right. This helper checks each of the four candidates and keeps only the ones that are inside the grid and are not walls. Both BFS and DFS will call this same function.

```
def neighbors(cell, maze):
    row, col = cell
    candidates = [
        (row - 1, col),    # up
        (row + 1, col),    # down
        (row, col - 1),    # left
        (row, col + 1)     # right
    ]
```

```

valid = []
for r, c in candidates:
    inside_grid = 0 <= r < len(maze) and 0 <= c < len(maze[0])
    if inside_grid and maze[r][c] == 0:
        valid.append((r, c))
return valid

```

Line by line:

Code	What it does
<code>row, col = cell</code>	Unpacks the (row, column) tuple
<code>0 <= r < len(maze)</code>	Checks the row is not off the top or bottom edge
<code>maze[r][c] == 0</code>	Checks the cell is walkable, not a wall
<code>valid.append(...)</code>	Adds the cell to the list we hand back

Step 3 — Rebuild the path once you reach the goal

While searching, we record for each cell which cell we came from, in a dictionary called `parent`. To get the path, we start at the goal and follow those links backwards to the start, then reverse the list.

```

def reconstruct_path(parent, start, goal):
    path = [goal]
    while path[-1] != start:
        path.append(parent[path[-1]])
    path.reverse()
    return path

```

Line by line:

Code	What it does
<code>path[-1]</code>	The last item in the list — negative indexing counts from the end
<code>parent[path[-1]]</code>	Looks up which cell we arrived from
<code>path.reverse()</code>	Flips goal-to-start round into start-to-goal

Step 4 — Breadth-First Search

The frontier is our to-do list. BFS uses `popleft()`, which removes from the FRONT — so cells discovered earliest are explored earliest, giving the level-by-level ripple effect. `visited` stops us checking the same cell twice and looping forever.

```

from collections import deque

def bfs(maze, start, goal):
    frontier = deque([start])    # the to-do list
    visited = {start}           # cells already seen (a set)
    parent = {}                 # remembers how we reached each cell

    while frontier:
        current = frontier.popleft()    # <-- TAKE FROM THE FRONT

        if current == goal:
            return reconstruct_path(parent, start, goal), visited

        for nxt in neighbors(current, maze):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = current

```

```

        frontier.append(nxt)

    return None, visited    # no path exists

```

Line by line:

Code	What it does
<code>deque</code>	A double-ended queue — lets us efficiently remove from either end
<code>popleft()</code>	Removes from the FRONT. THIS single call is what makes it BFS
<code>while frontier:</code>	Keep going while there is anything left to explore
<code>visited = {start}</code>	Curly braces with no colons creates a SET — fast membership checking
<code>parent[nxt] = current</code>	Records the breadcrumb trail for path reconstruction

Step 5 — Depth-First Search (change one line!)

Compare this carefully with Step 4. Everything is identical except `pop()` instead of `popleft()`. `pop()` removes from the BACK, so the most recently discovered cell is explored next — diving deep instead of spreading wide.

```

def dfs(maze, start, goal):
    frontier = [start]          # a plain list works as a stack
    visited = {start}
    parent = {}

    while frontier:
        current = frontier.pop()    # <-- TAKE FROM THE BACK

        if current == goal:
            return reconstruct_path(parent, start, goal), visited

        for nxt in neighbors(current, maze):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = current
                frontier.append(nxt)

    return None, visited

```

Line by line:

Code	What it does
<code>pop()</code>	Removes from the BACK. This one change turns BFS into DFS
<code>frontier = [start]</code>	A plain list is enough, since we only add and remove at one end
Everything else	Character-for-character identical to BFS

Step 6 — Draw the result

We convert the maze into a picture where walls, explored cells and the final path each get a different shade, then save it as a PNG file you can open.

```

import matplotlib.pyplot as plt
import numpy as np

def visualize(maze, path, visited, title, filename):
    grid = np.array(maze, dtype=float)

    for cell in visited:

```

```

    grid[cell] = 0.4      # explored but not on the final path
    for cell in path:
        grid[cell] = 0.8  # the final path

plt.figure()
plt.imshow(grid, cmap='viridis')
plt.title(title)
plt.savefig(filename)
plt.close()
print('Saved', filename)

```

Line by line:

Code	What it does
<code>np.array(maze, dtype=float)</code>	Converts to a NumPy grid holding decimals, so we can use in-between shades
<code>grid[cell] = 0.4</code>	cell is a (row, col) tuple, which NumPy accepts directly as an index
<code>plt.imshow</code>	Draws a 2D array as a coloured image
<code>plt.close()</code>	Frees the figure so the next chart starts fresh

The complete program (lab3.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab3.py – BFS and DFS maze pathfinding
from collections import deque
import matplotlib.pyplot as plt
import numpy as np

maze = [
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]
start = (0, 0)
goal = (4, 4)

def neighbors(cell, maze):
    row, col = cell
    candidates = [(row-1,col), (row+1,col), (row,col-1), (row,col+1)]
    valid = []
    for r, c in candidates:
        if 0 <= r < len(maze) and 0 <= c < len(maze[0]) and maze[r][c] == 0:
            valid.append((r, c))
    return valid

def reconstruct_path(parent, start, goal):
    path = [goal]
    while path[-1] != start:
        path.append(parent[path[-1]])
    path.reverse()
    return path

def bfs(maze, start, goal):
    frontier = deque([start])
    visited = {start}
    parent = {}
    while frontier:
        current = frontier.popleft()
        if current == goal:
            return reconstruct_path(parent, start, goal), visited
        for nxt in neighbors(current, maze):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = current
                frontier.append(nxt)
    return None, visited

def dfs(maze, start, goal):
    frontier = [start]
    visited = {start}
    parent = {}
    while frontier:
        current = frontier.pop()
        if current == goal:
            return reconstruct_path(parent, start, goal), visited
        for nxt in neighbors(current, maze):
            if nxt not in visited:
```

```

        visited.add(nxt)
        parent[nxt] = current
        frontier.append(nxt)
    return None, visited

def visualize(maze, path, visited, title, filename):
    grid = np.array(maze, dtype=float)
    for cell in visited:
        grid[cell] = 0.4
    for cell in path:
        grid[cell] = 0.8
    plt.figure()
    plt.imshow(grid, cmap='viridis')
    plt.title(title)
    plt.savefig(filename)
    plt.close()
    print('Saved', filename)

bfs_path, bfs_visited = bfs(maze, start, goal)
dfs_path, dfs_visited = dfs(maze, start, goal)

print('BFS path length:', len(bfs_path), '| cells explored:', len(bfs_visited))
print('DFS path length:', len(dfs_path), '| cells explored:', len(dfs_visited))
print()
print('BFS path:', bfs_path)
print('DFS path:', dfs_path)

visualize(maze, bfs_path, bfs_visited, 'BFS Result', 'lab3_bfs.png')
visualize(maze, dfs_path, dfs_visited, 'DFS Result', 'lab3_dfs.png')

```

HOW TO RUN IT

```
python lab3.py
```

WHAT YOU SHOULD SEE

```

BFS path length: 9 | cells explored: 16
DFS path length: 9 | cells explored: 13

BFS path: [(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), (4,4)]
DFS path: [(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), (4,4)]

Saved lab3_bfs.png
Saved lab3_dfs.png

```

(On this small maze both happen to find the same length path. On larger mazes DFS will usually return a LONGER path than BFS. Open the two PNG files to see the difference in which cells each algorithm explored.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
KeyError when reconstructing the path	The goal was never reached, so parent has no entry for it	Check that a path actually exists; print the returned path before using it
Program hangs forever	You forgot the visited set, so the search revisits cells in a loop	Always add a cell to visited at the moment you push it onto the frontier
IndexError: list index out of range	Bounds checking is missing or wrong in neighbors()	Verify the condition: $0 \leq r < \text{len}(\text{maze})$ and $0 \leq c < \text{len}(\text{maze}[0])$
Blank or all-one-colour image	path and visited contain no cells, or all values are identical	Print $\text{len}(\text{visited})$ to confirm the search actually ran

TRY IT YOURSELF

- Change only `popleft()` to `pop()` in `bfs()` and confirm it behaves like DFS.
- Build a bigger 10x10 maze and compare the path lengths again — now the difference should appear.
- Print the order in which cells are explored to see the ripple versus dive pattern.
- Block the maze completely with walls and confirm both functions return `None` gracefully.

LAB 4: Compare Performance of Uninformed vs. Informed Search

*Relates to: Weeks 4–5 — Evaluating Search Strategies, Heuristics, A**

WHAT THIS LAB IS ABOUT

BFS is blind: it treats every direction as equally promising and spreads out evenly in all directions. A* is informed: it gets a hint, called a heuristic, estimating how far each cell is from the goal, and uses that hint to explore towards the goal first. The formula A* uses is $f = g + h$, where g is the real cost of getting to a cell so far, and h is the estimated cost remaining. In this lab you count how many cells each algorithm examines, proving with numbers that the hint saves work.

BEFORE YOU START, YOU NEED

- Completion of Lab 3 (you will reuse the maze and the neighbors function)
- Understanding of Python's `heapq` module (explained below)

Building it step by step

Step 1 — Write the heuristic

Manhattan distance counts how many steps you would need if there were no walls at all: the horizontal gap plus the vertical gap. It is called admissible because it never overestimates — walls can only make the real journey longer, never shorter. Admissibility is what guarantees A* still finds the shortest path.

```
def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

# Example:
print(manhattan((0,0), (4,4))) # 4 + 4 = 8
print(manhattan((2,3), (4,4))) # 2 + 1 = 3
```

Line by line:

Code	What it does
<code>abs(...)</code>	Absolute value — turns a negative difference positive
<code>a[0] - b[0]</code>	The vertical (row) gap
<code>a[1] - b[1]</code>	The horizontal (column) gap
Never overestimates	Because ignoring walls can only make the estimate too SMALL, never too big

Step 2 — Add counters to BFS

To compare fairly we need a number. We add a variable called `expanded` that increases by one every time we take a cell off the frontier and examine it. This is the standard measure of search effort.

```
from collections import deque

def bfs_counted(maze, start, goal):
    frontier = deque([start])
    visited = {start}
    parent = {}
    expanded = 0 # <-- the counter

    while frontier:
        current = frontier.popleft()
        expanded += 1 # <-- count this examination

        if current == goal:
            path = reconstruct_path(parent, start, goal)
```

```

        return path, expanded

    for nxt in neighbors(current, maze):
        if nxt not in visited:
            visited.add(nxt)
            parent[nxt] = current
            frontier.append(nxt)

    return None, expanded

```

Line by line:

Code	What it does
<code>expanded = 0</code>	Starts the tally at zero
<code>expanded += 1</code>	Shorthand for <code>expanded = expanded + 1</code>
<code>return path, expanded</code>	Returns two things at once, as a tuple

Step 3 — Understand the priority queue

A* must always examine the most promising cell next, not the oldest or newest. A priority queue does exactly that: no matter what order you put things in, `heappop` always hands back the item with the smallest first value. Try this snippet on its own to see it work.

```

import heapq

queue = []
heapq.heappush(queue, (5, 'expensive'))
heapq.heappush(queue, (1, 'cheap'))
heapq.heappush(queue, (3, 'medium'))

print(heapq.heappop(queue)) # (1, 'cheap') <- smallest comes out first
print(heapq.heappop(queue)) # (3, 'medium')
print(heapq.heappop(queue)) # (5, 'expensive')

```

Line by line:

Code	What it does
<code>heapq.heappush(queue, item)</code>	Adds an item, keeping the queue ordered
<code>heapq.heappop(queue)</code>	Removes and returns the SMALLEST item
<code>(5, 'expensive')</code>	A tuple; <code>heapq</code> sorts by the first element

Step 4 — Write A* search

Each frontier entry is `(f, g, cell)`. We always pop the lowest `f`. `g_cost` remembers the cheapest known cost to reach each cell, so if we find a better route to a cell we already saw, we update it. That check is what keeps A* correct.

```

import heapq

def a_star(maze, start, goal):
    # Each entry is (f_score, g_score, cell)
    frontier = [(manhattan(start, goal), 0, start)]
    g_cost = {start: 0}
    parent = {}
    expanded = 0

    while frontier:

```

```

f, g, current = heapq.heappop(frontier)
expanded += 1

if current == goal:
    path = reconstruct_path(parent, start, goal)
    return path, expanded

for nxt in neighbors(current, maze):
    new_g = g + 1          # every step costs 1

    # Only continue if this is the cheapest route found so far
    if nxt not in g_cost or new_g < g_cost[nxt]:
        g_cost[nxt] = new_g
        f_score = new_g + manhattan(nxt, goal)
        heapq.heappush(frontier, (f_score, new_g, nxt))
        parent[nxt] = current

return None, expanded

```

Line by line:

Code	What it does
(manhattan(start,goal), 0, start)	At the start: g is 0, so f equals h
f, g, current = heapq.heappop(...)	Unpacks the three values from the lowest-f entry
new_g = g + 1	The real cost to reach the neighbour: one more step
f_score = new_g + manhattan(nxt, goal)	THE A* FORMULA: $f = g + h$
new_g < g_cost[nxt]	We found a cheaper route to a cell we already knew about

Step 5 — Time both and print a comparison table

`time.perf_counter()` gives a high-precision timestamp. Subtracting two of them gives the elapsed seconds; we multiply by 1000 to show milliseconds.

```

import time

t0 = time.perf_counter()
bfs_path, bfs_expanded = bfs_counted(maze, start, goal)
t1 = time.perf_counter()

t2 = time.perf_counter()
astar_path, astar_expanded = a_star(maze, start, goal)
t3 = time.perf_counter()

print(f"{'Algorithm':<12}{'PathLen':<10}{'Expanded':<12}{'Time(ms)':<10}")
print('-' * 44)
print(f"{'BFS':<12}{len(bfs_path):<10}{bfs_expanded:<12}{(t1-t0)*1000:<10.3f}")
print(f"{'A*':<12}{len(astar_path):<10}{astar_expanded:<12}{(t3-t2)*1000:<10.3f}")

saving = 100 * (1 - astar_expanded / bfs_expanded)
print()
print(f'A* examined {saving:.1f}% fewer cells than BFS.')

```

Line by line:

Code	What it does
<code>time.perf_counter()</code>	A precise stopwatch reading, in seconds
<code>f"..."</code>	An f-string: anything inside curly braces is evaluated and inserted

Code	What it does
<code>{ 'BFS' :<12}</code>	Pads the text to 12 characters wide, left-aligned, making columns line up
<code>{ (t1-t0)*1000:<10.3f}</code>	Shows the number to 3 decimal places in a 10-wide column

The complete program (lab4.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab4.py – Comparing BFS against A*
from collections import deque
import heapq
import time

maze = [
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]
start = (0, 0)
goal = (4, 4)

def neighbors(cell, maze):
    row, col = cell
    candidates = [(row-1,col), (row+1,col), (row,col-1), (row,col+1)]
    valid = []
    for r, c in candidates:
        if 0 <= r < len(maze) and 0 <= c < len(maze[0]) and maze[r][c] == 0:
            valid.append((r, c))
    return valid

def reconstruct_path(parent, start, goal):
    path = [goal]
    while path[-1] != start:
        path.append(parent[path[-1]])
    path.reverse()
    return path

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def bfs_counted(maze, start, goal):
    frontier = deque([start])
    visited = {start}
    parent = {}
    expanded = 0
    while frontier:
        current = frontier.popleft()
        expanded += 1
        if current == goal:
            return reconstruct_path(parent, start, goal), expanded
        for nxt in neighbors(current, maze):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = current
                frontier.append(nxt)
    return None, expanded

def a_star(maze, start, goal):
    frontier = [(manhattan(start, goal), 0, start)]
    g_cost = {start: 0}
    parent = {}
    expanded = 0
```

```

while frontier:
    f, g, current = heapq.heappop(frontier)
    expanded += 1
    if current == goal:
        return reconstruct_path(parent, start, goal), expanded
    for nxt in neighbors(current, maze):
        new_g = g + 1
        if nxt not in g_cost or new_g < g_cost[nxt]:
            g_cost[nxt] = new_g
            f_score = new_g + manhattan(nxt, goal)
            heapq.heappush(frontier, (f_score, new_g, nxt))
            parent[nxt] = current
    return None, expanded

t0 = time.perf_counter()
bfs_path, bfs_expanded = bfs_counted(maze, start, goal)
t1 = time.perf_counter()

t2 = time.perf_counter()
astar_path, astar_expanded = a_star(maze, start, goal)
t3 = time.perf_counter()

print(f"{'Algorithm':<12}{'PathLen':<10}{'Expanded':<12}{'Time(ms)':<10}")
print('-' * 44)
print(f"{'BFS':<12}{len(bfs_path):<10}{bfs_expanded:<12}{(t1-t0)*1000:<10.3f}")
print(f"{'A*':<12}{len(astar_path):<10}{astar_expanded:<12}{(t3-t2)*1000:<10.3f}")

saving = 100 * (1 - astar_expanded / bfs_expanded)
print()
print(f'A* examined {saving:.1f}% fewer cells than BFS.')
print('Both found a path of the same length, so A* lost no quality.')

```

HOW TO RUN IT

```
python lab4.py
```

WHAT YOU SHOULD SEE

Algorithm	PathLen	Expanded	Time(ms)
BFS	9	16	0.021
A*	9	11	0.018

A* examined 31.2% fewer cells than BFS.
Both found a path of the same length, so A* lost no quality.

(Exact timings vary by machine. The key result is the Expanded column:
A* should always be lower than or equal to BFS, while PathLen stays
identical – proving the heuristic saved work without costing quality.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
TypeError: '<' not supported between instances of 'tuple' and 'tuple'	Two frontier entries had identical f and g, so Python tried to compare the cells	Add a counter as a third tie-breaker value, as shown in Lab 5
A* returns a longer path than BFS	The heuristic overestimates — it is not admissible	Make sure you use Manhattan distance, not Euclidean squared or a scaled version
Both report the same Expanded count	The maze is too small or too open for the heuristic to help	Try a larger maze with a longer, more winding corridor
ZeroDivisionError computing saving	bfs_expanded was zero, meaning the	Check that start and goal are both on walkable

Error message / symptom	Why it happened	How to fix it
	search never ran	(0) cells

TRY IT YOURSELF

- Set the heuristic to always return 0 and re-run. A* should now expand exactly as many cells as BFS — because with no hint, A* degenerates into uninformed search.
- Multiply the heuristic by 3. The path may get longer, showing what happens when a heuristic is not admissible.
- Build a 20x20 maze and see how much larger the saving becomes.
- Add Greedy Best-First Search (use only h, ignoring g) and add it as a third row in the table.

LAB 5: Implement A* Search to Solve the 8-Puzzle Problem

*Relates to: Week 5 — Informed Search: A**

WHAT THIS LAB IS ABOUT

The 8-puzzle is the sliding tile game: a 3x3 frame holds eight numbered tiles and one empty gap. You slide tiles into the gap until the numbers are in order. It looks trivial but there are 181,440 reachable arrangements, so guessing is hopeless. This is where A* earns its keep. The key mental shift from Lab 4 is that a 'position' is no longer a coordinate on a grid — it is an entire board arrangement. Moving is no longer stepping north; it is sliding a tile.

BEFORE YOU START, YOU NEED

- Completion of Lab 4 (you must understand $f = g + h$ and `heapq`)
- Understanding of Python tuples and the `divmod` function
- No external libraries needed

Building it step by step

Step 1 — Represent a board as a flat tuple

Instead of a 3x3 nested list, we flatten the board into nine values in a single tuple, reading left to right, top to bottom. The number 0 represents the empty gap. We use a tuple rather than a list because tuples cannot be changed, which lets Python use them as dictionary keys — essential for tracking visited states.

```
# The board:      As a flat tuple:
# 1 2 3
# 4 5 6    -->   (1, 2, 3, 4, 5, 6, 7, 8, 0)
# 7 8 _

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

start_state = (1, 2, 3, 4, 0, 6, 7, 5, 8)

def print_board(state):
    for row in range(3):
        line = state[row*3 : row*3 + 3]
        print(' '.join('_' if v == 0 else str(v) for v in line))

print_board(start_state)
```

Line by line:

Code	What it does
<code>(1,2,3,4,5,6,7,8,0)</code>	A tuple — round brackets, contents cannot be modified
<code>0</code>	Represents the empty gap
<code>state[row*3 : row*3+3]</code>	Slices out three values, giving one row of the board
Why a tuple, not a list?	Tuples are hashable, so they can be dictionary keys and set members

Step 2 — Convert between flat position and row/column

Position 4 in a flat tuple is row 1, column 1 on the board. `divmod` does both divisions at once: `divmod(4, 3)` gives (1, 1). You will use this constantly.

```
# Flat index -> (row, column)
print(divmod(0, 3)) # (0, 0) top-left
print(divmod(4, 3)) # (1, 1) centre
print(divmod(8, 3)) # (2, 2) bottom-right
```

```
# (row, column) -> flat index
row, col = 1, 1
flat = row * 3 + col
print(flat)          # 4
```

Line by line:

Code	What it does
<code>divmod(a, b)</code>	Returns (a // b, a % b) — the quotient and the remainder together
<code>row * 3 + col</code>	The reverse conversion, back to a flat index

Step 3 — Write the heuristic

Manhattan distance for the 8-puzzle means: for every tile, how many slides away is it from where it belongs? Add all those up. We skip the gap (0) because moving the gap is not the goal. This never overestimates, because each slide can improve one tile's distance by at most one.

```
def manhattan_distance(state):
    distance = 0

    for index, tile in enumerate(state):
        if tile == 0:
            continue          # skip the empty gap

        goal_index = GOAL.index(tile)

        current_row, current_col = divmod(index, 3)
        goal_row, goal_col = divmod(goal_index, 3)

        distance += abs(current_row - goal_row) + abs(current_col - goal_col)

    return distance

print('Heuristic for start state:', manhattan_distance(start_state))
```

Line by line:

Code	What it does
<code>enumerate(state)</code>	Loops giving both the position number and the value at it
<code>continue</code>	Skip the rest of this loop iteration and move to the next tile
<code>GOAL.index(tile)</code>	Finds where this tile SHOULD be in the finished puzzle
<code>abs(...) + abs(...)</code>	Vertical distance plus horizontal distance for this one tile

Step 4 — Generate the legal moves

From any board, the gap can slide up, down, left or right — unless it is against an edge. We find where the gap is, try all four directions, and keep the legal ones. Each result is a brand new board plus the name of the move that produced it.

```
def get_successors(state):
    successors = []

    blank = state.index(0)
    row, col = divmod(blank, 3)

    moves = [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1, 'Right')]
```

```

for delta_row, delta_col, name in moves:
    new_row = row + delta_row
    new_col = col + delta_col

    # Skip moves that would fall off the board
    if not (0 <= new_row < 3 and 0 <= new_col < 3):
        continue

    new_blank = new_row * 3 + new_col

    # Swap the gap with the tile next to it
    new_state = list(state)
    new_state[blank], new_state[new_blank] = new_state[new_blank], new_state[blank]

    successors.append((tuple(new_state), name))

return successors

for board, move in get_successors(start_state):
    print(move, '->', board)

```

Line by line:

Code	What it does
<code>state.index(0)</code>	Finds the flat position of the empty gap
<code>(-1, 0, 'Up')</code>	Moving up means row decreases by 1, column unchanged
<code>list(state)</code>	Converts the fixed tuple into a changeable list so we can swap
<code>a, b = b, a</code>	Python's swap idiom — exchanges two values in one line
<code>tuple(new_state)</code>	Converts back to a tuple so it can be used as a dictionary key

Step 5 — Run A* over board states

This is structurally the same A* as Lab 4. The only differences: states are board tuples instead of coordinates, and we carry the list of moves along with each entry so we can report the solution. The counter variable breaks ties when two boards have identical f and g, preventing Python from trying to compare tuples.

```

import heapq

def a_star_puzzle(start):
    counter = 0
    # Each entry: (f, g, tie_breaker, board, moves_so_far)
    frontier = [(manhattan_distance(start), 0, counter, start, [])]
    best_g = {start: 0}
    expanded = 0

    while frontier:
        f, g, _, state, path = heapq.heappop(frontier)
        expanded += 1

        if state == GOAL:
            return path, expanded

        for new_state, move in get_successors(state):
            new_g = g + 1

            if new_state not in best_g or new_g < best_g[new_state]:

```

```

        best_g[new_state] = new_g
        counter += 1
        h = manhattan_distance(new_state)
        heapq.heappush(
            frontier,
            (new_g + h, new_g, counter, new_state, path + [move])
        )

    return None, expanded

```

Line by line:

Code	What it does
counter	A unique increasing number that breaks ties so heapq never compares boards
—	A conventional name for a value we must unpack but do not use
path + [move]	Creates a NEW list with this move appended — does not modify the old one
best_g	Records the cheapest number of moves found to reach each board so far
new_g + h	The A* formula again: $f = g + h$

The complete program (lab5.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab5.py – Solving the 8-puzzle with A*
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def print_board(state):
    for row in range(3):
        line = state[row*3 : row*3 + 3]
        print(' '.join('_' if v == 0 else str(v) for v in line))

def manhattan_distance(state):
    distance = 0
    for index, tile in enumerate(state):
        if tile == 0:
            continue
        goal_index = GOAL.index(tile)
        current_row, current_col = divmod(index, 3)
        goal_row, goal_col = divmod(goal_index, 3)
        distance += abs(current_row - goal_row) + abs(current_col - goal_col)
    return distance

def get_successors(state):
    successors = []
    blank = state.index(0)
    row, col = divmod(blank, 3)
    moves = [(-1,0,'Up'), (1,0,'Down'), (0,-1,'Left'), (0,1,'Right')]
    for delta_row, delta_col, name in moves:
        new_row, new_col = row + delta_row, col + delta_col
        if not (0 <= new_row < 3 and 0 <= new_col < 3):
            continue
        new_blank = new_row * 3 + new_col
        new_state = list(state)
        new_state[blank], new_state[new_blank] = new_state[new_blank], new_state[blank]
        successors.append((tuple(new_state), name))
    return successors

def a_star_puzzle(start):
    counter = 0
    frontier = [(manhattan_distance(start), 0, counter, start, [])]
    best_g = {start: 0}
    expanded = 0
    while frontier:
        f, g, _, state, path = heapq.heappop(frontier)
        expanded += 1
        if state == GOAL:
            return path, expanded
        for new_state, move in get_successors(state):
            new_g = g + 1
            if new_state not in best_g or new_g < best_g[new_state]:
                best_g[new_state] = new_g
                counter += 1
                h = manhattan_distance(new_state)
                heapq.heappush(frontier,
                               (new_g + h, new_g, counter, new_state, path + [move]))
    return None, expanded
```

```

start_state = (1, 2, 3, 4, 0, 6, 7, 5, 8)

print('Starting board:')
print_board(start_state)
print()
print('Heuristic estimate:', manhattan_distance(start_state))
print()

solution, nodes_expanded = a_star_puzzle(start_state)

print('Solution moves:', solution)
print('Number of moves:', len(solution))
print('Boards examined:', nodes_expanded)
print()
print('Goal board:')
print_board(GOAL)

```

HOW TO RUN IT

```
python lab5.py
```

WHAT YOU SHOULD SEE

```

Starting board:
1 2 3
4 _ 6
7 5 8

Heuristic estimate: 2

Solution moves: ['Down', 'Right']
Number of moves: 2
Boards examined: 3

Goal board:
1 2 3
4 5 6
7 8 _

(This start is deliberately easy. Try a harder scramble and watch the
'Boards examined' number grow dramatically – that growth is exactly why
a good heuristic matters.)

```

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
<code>TypeError: '<' not supported between instances of 'tuple' and 'tuple'</code>	Two entries tied on f and g, so heapq tried to compare the board tuples	Add the counter tie-breaker as the third element, as shown
<code>TypeError: unhashable type: 'list'</code>	You used a list instead of a tuple as a dictionary key	Always convert with <code>tuple(...)</code> before storing in <code>best_g</code>
Program runs forever	The puzzle you entered is unsolvable — half of all arrangements are	Start from <code>GOAL</code> and make a few random legal moves to guarantee solvability
<code>ValueError: 0 is not in list</code>	The board is missing the empty gap	Make sure exactly one of your nine values is 0

TRY IT YOURSELF

- Try a harder start such as (4, 1, 3, 7, 2, 6, 0, 5, 8) and compare the boards examined.
- Replace the heuristic with one that always returns 0 and observe the explosion in nodes examined — this is A* reduced to uninformed search.

- Write a 'misplaced tiles' heuristic (count how many tiles are not in the right place) and compare its efficiency against Manhattan distance.
- Print each intermediate board along the solution path.

LAB 6: Implement Hill Climbing; Analyze Its Tendency to Get Stuck

Relates to: Week 6 — Local Search: Hill Climbing, Beam Search

WHAT THIS LAB IS ABOUT

Every search so far kept a list of places to explore. Hill climbing throws that away. It holds exactly one candidate solution, looks at every small change it could make, takes the single best one, and repeats — stopping when no change improves things. It is fast and uses almost no memory. But it is also short-sighted: if it reaches a spot where every neighbouring change makes things worse, it stops — even though a much better solution exists elsewhere. That trap is called a local optimum, and this lab measures exactly how often it happens.

BEFORE YOU START, YOU NEED

- Completion of Labs 1–2
- Understanding of Python lists and nested loops
- Only the built-in random module

Building it step by step

Step 1 — Represent a board of eight queens

The 8-Queens puzzle asks you to place eight queens on a chessboard so none can attack another. We use a clever compact representation: a list of eight numbers, where position *i* holds the row of the queen in column *i*. This automatically prevents two queens sharing a column, removing one whole class of conflict for free.

```
import random

N = 8

# state[column] = the row that column's queen sits in
example = [0, 4, 7, 5, 2, 6, 1, 3]

def show_board(state):
    for row in range(N):
        line = ''
        for col in range(N):
            line += 'Q ' if state[col] == row else '. '
        print(line)

show_board(example)
```

Line by line:

Code	What it does
<code>state[col] == row</code>	True if this column's queen is in this row — so draw a Q
One number per column	Guarantees exactly one queen per column, automatically
Values 0 to 7	Which row that queen occupies

Step 2 — Count how many pairs of queens attack each other

Two queens attack each other if they share a row, or sit on the same diagonal. The diagonal test is elegant: if the vertical gap equals the horizontal gap, they are diagonal. We check every pair exactly once. Zero attacking pairs means a solution.

```
def attacking_pairs(state):
    count = 0

    for i in range(N):
        for j in range(i + 1, N):      # j starts after i, so each pair once
```

```

        same_row = state[i] == state[j]
        same_diagonal = abs(state[i] - state[j]) == abs(i - j)

        if same_row or same_diagonal:
            count += 1

    return count

print('Attacking pairs in example:', attacking_pairs(example))

```

Line by line:

Code	What it does
for j in range(i+1, N)	Starting at i+1 avoids counting the same pair twice
state[i] == state[j]	Same row means they attack horizontally
abs(row gap) == abs(col gap)	The diagonal test — equal gaps means same diagonal
Returns 0	Means a perfect solution: no queen attacks any other

Step 3 — Generate all the small changes you could make

A neighbour is any board reachable by moving exactly one queen to a different row in its own column. With 8 columns and 7 alternative rows each, there are 56 neighbours. Hill climbing will examine all of them and pick the best.

```

def get_neighbors(state):
    neighbors = []

    for col in range(N):
        for row in range(N):
            if row == state[col]:
                continue                # that is the current position, not a change

            new_state = list(state)    # make a copy
            new_state[col] = row       # move this one queen
            neighbors.append(new_state)

    return neighbors

print('Number of neighbours:', len(get_neighbors(example)))    # 56

```

Line by line:

Code	What it does
list(state)	Makes a COPY — without this you would corrupt the original board
new_state[col] = row	Move one queen, leaving all others untouched
56 neighbours	8 columns x 7 alternative rows

Step 4 — The hill climbing loop

Look at all neighbours, find the one with the fewest attacking pairs, and move there. If the best neighbour is no better than where we already are, stop — we are at the top of a hill. Crucially, that hill might not be the highest hill.

```

def hill_climb(state):
    current = state
    current_cost = attacking_pairs(current)

```

```

while True:
    neighbors = get_neighbors(current)
    best_neighbor = min(neighbors, key=attacking_pairs)
    best_cost = attacking_pairs(best_neighbor)

    # No neighbour is an improvement -> we are stuck at a peak
    if best_cost >= current_cost:
        return current, current_cost

    current = best_neighbor
    current_cost = best_cost

```

Line by line:

Code	What it does
while True:	Loop forever, until an explicit return breaks out
min(neighbors, key=attacking_pairs)	Finds the neighbour with the SMALLEST attacking_pairs value
best_cost >= current_cost	Nothing improves things — stop here
Returns cost 0	A true solution. Any other number means it got stuck

Step 5 — Measure how often it fails

Run hill climbing from 100 random starting boards and count how many reach a real solution. This is the experiment that makes the local-optima problem concrete rather than theoretical.

```

def random_state():
    return [random.randint(0, N-1) for _ in range(N)]

random.seed(1)

successes = 0
stuck = 0
stuck_costs = []

for trial in range(100):
    result, cost = hill_climb(random_state())
    if cost == 0:
        successes += 1
    else:
        stuck += 1
        stuck_costs.append(cost)

print(f'Solved:          {successes}/100')
print(f'Stuck in a local optimum: {stuck}/100')
print(f'Average conflicts when stuck: {sum(stuck_costs)/len(stuck_costs):.2f}')

```

Line by line:

Code	What it does
[... for _ in range(N)]	A list comprehension — builds a list of 8 random rows
random.seed(1)	Makes the experiment repeatable
cost == 0	A genuine solution was found
stuck_costs	Records HOW stuck it was — usually only 1 or 2 conflicts remain

Step 6 — Fix it with random restarts

The cure is embarrassingly simple: if you get stuck, throw the board away and start again somewhere else. Since each attempt is fast and succeeds about 14% of the time, trying repeatedly almost always finds a solution.

```
def random_restart_hill_climb(max_restarts=100):
    for attempt in range(max_restarts):
        result, cost = hill_climb(random_state())
        if cost == 0:
            return result, cost, attempt + 1
    return result, cost, max_restarts

random.seed(1)
solution, cost, attempts_needed = random_restart_hill_climb()

print()
print('Random-restart hill climbing:')
print('  Conflicts:', cost)
print('  Restarts needed:', attempts_needed)
print()
show_board(solution)
```

Line by line:

Code	What it does
<code>for attempt in range(max_restarts)</code>	Try up to 100 fresh starting boards
<code>if cost == 0: return</code>	Stop the moment we find a real solution
<code>attempt + 1</code>	Human-friendly count, since range starts at 0

The complete program (lab6.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab6.py – Hill climbing and local optima
import random

N = 8

def show_board(state):
    for row in range(N):
        line = ''
        for col in range(N):
            line += 'Q ' if state[col] == row else '. '
        print(line)

def attacking_pairs(state):
    count = 0
    for i in range(N):
        for j in range(i + 1, N):
            if state[i] == state[j] or abs(state[i]-state[j]) == abs(i-j):
                count += 1
    return count

def get_neighbors(state):
    neighbors = []
    for col in range(N):
        for row in range(N):
            if row == state[col]:
                continue
            new_state = list(state)
            new_state[col] = row
            neighbors.append(new_state)
    return neighbors

def hill_climb(state):
    current = state
    current_cost = attacking_pairs(current)
    while True:
        neighbors = get_neighbors(current)
        best_neighbor = min(neighbors, key=attacking_pairs)
        best_cost = attacking_pairs(best_neighbor)
        if best_cost >= current_cost:
            return current, current_cost
        current, current_cost = best_neighbor, best_cost

def random_state():
    return [random.randint(0, N-1) for _ in range(N)]

def random_restart_hill_climb(max_restarts=100):
    for attempt in range(max_restarts):
        result, cost = hill_climb(random_state())
        if cost == 0:
            return result, cost, attempt + 1
    return result, cost, max_restarts

# --- Experiment 1: how often does plain hill climbing succeed? ---
random.seed(1)
successes, stuck, stuck_costs = 0, 0, []
```

```

for trial in range(100):
    result, cost = hill_climb(random_state())
    if cost == 0:
        successes += 1
    else:
        stuck += 1
        stuck_costs.append(cost)

print('=== Plain Hill Climbing (100 random starts) ===')
print(f'Solved:                {successes}/100')
print(f'Stuck in local optimum: {stuck}/100')
print(f'Avg conflicts when stuck: {sum(stuck_costs)/len(stuck_costs):.2f}')
print()

# --- Experiment 2: does restarting fix it? ---
random.seed(1)
solution, cost, attempts_needed = random_restart_hill_climb()

print('=== Random-Restart Hill Climbing ===')
print('Conflicts remaining:', cost)
print('Restarts needed:      ', attempts_needed)
print()
print('Solution board:')
show_board(solution)

```

HOW TO RUN IT

```
python lab6.py
```

WHAT YOU SHOULD SEE

```

=== Plain Hill Climbing (100 random starts) ===
Solved:                14/100
Stuck in local optimum: 86/100
Avg conflicts when stuck: 1.23

```

```

=== Random-Restart Hill Climbing ===
Conflicts remaining: 0
Restarts needed:      4

```

Solution board:

```

. . . . Q . . .
. Q . . . . .
. . . Q . . .
Q . . . . .
. . . . . Q .
. . . . . Q
. . . . Q .
. . Q . . .

```

(The famous result: plain hill climbing solves only about 14% of random starts. Notice that when it fails it is usually only 1 or 2 conflicts away – agonisingly close, but unable to get there.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
Success rate is 0/100	attacking_pairs has a bug — probably the diagonal test	Verify: <code>abs(state[i]-state[j]) == abs(i-j)</code> , not <code>== (i-j)</code>
Program never stops	The stop condition uses <code>></code> instead of <code>>=</code> , so it loops on equal-cost neighbours	Use: if <code>best_cost >= current_cost</code> : return

Error message / symptom	Why it happened	How to fix it
All boards come out identical	You mutated state instead of copying it in <code>get_neighbors</code>	Always use <code>new_state = list(state)</code> before changing anything
<code>ZeroDivisionError</code> on the average	Hill climbing solved all 100, so <code>stuck_costs</code> is empty	Guard with: <code>if stuck_costs else 0</code>

TRY IT YOURSELF

- Change `N` to 4 and re-run. Is 4-Queens easier or harder to solve by hill climbing?
- Add sideways moves: allow up to 100 equal-cost moves before giving up, and see if the success rate improves.
- Implement first-choice hill climbing: take the FIRST improving neighbour found rather than checking all 56.
- Record how many steps hill climbing takes before getting stuck, and plot the distribution.

LAB 7: Develop a Tic-Tac-Toe Game with the Min-Max Algorithm

Relates to: Week 7 — Adversarial Search, Min-Max

WHAT THIS LAB IS ABOUT

Everything so far assumed the world sits still while you think. Games do not: there is an opponent actively working against you. Min-Max handles this by assuming the worst — that your opponent always plays the best possible move. You (MAX) pick moves that maximise your score; the opponent (MIN) picks moves that minimise it. The algorithm plays out every possible continuation of the game to the end, then works backwards to decide what to do now. For Tic-Tac-Toe this is completely feasible, and the result is an opponent that literally cannot be beaten.

BEFORE YOU START, YOU NEED

- Completion of Labs 1–2
- Understanding of recursion (a function that calls itself) — explained in Step 3
- No external libraries

Building it step by step

Step 1 — Represent the board

Nine cells in a flat list, numbered 0 to 8 reading left to right, top to bottom. A space means empty. This is the same flattening trick as the 8-puzzle in Lab 5.

```
# 0 | 1 | 2
# -----
# 3 | 4 | 5
# -----
# 6 | 7 | 8

board = [' '] * 9      # nine empty cells

def print_board(board):
    for r in range(0, 9, 3):
        print(' ' + ' | '.join(board[r:r+3]))
        if r < 6:
            print(' -----')

board[4] = 'O'
print_board(board)
```

Line by line:

Code	What it does
<code>[' '] * 9</code>	Creates a list of nine identical space characters
<code>range(0, 9, 3)</code>	Counts 0, 3, 6 — the starting index of each row
<code>' '.join(...)</code>	Glues three cells together with separators between them

Step 2 — Detect the end of the game

There are exactly eight ways to win: three rows, three columns, two diagonals. We list them as index triples and check each. If nobody won and there are no empty cells left, it is a draw. If the game is still going, we return None.

```
def check_winner(board):
    lines = [
        (0,1,2), (3,4,5), (6,7,8),      # rows
        (0,3,6), (1,4,7), (2,5,8),      # columns
        (0,4,8), (2,4,6)                 # diagonals
```

```

]

for a, b, c in lines:
    if board[a] == board[b] == board[c] != ' ':
        return board[a]          # 'X' or 'O'

if ' ' not in board:
    return 'Draw'

return None                      # game still in progress

```

Line by line:

Code	What it does
<code>board[a] == board[b] == board[c]</code>	Python allows chained comparison — all three must match
<code>!= ' '</code>	And they must not all be empty, or every blank line would 'win'
<code>' ' not in board</code>	No empty cells remain
<code>return None</code>	The game is not over yet

Step 3 — Understand recursion first

Min-Max calls itself. If that idea is new, run this tiny example first. A recursive function needs two parts: a base case that stops the recursion, and a recursive case that calls itself on a smaller problem.

```

def countdown(n):
    if n == 0:          # BASE CASE: stop here
        print('Liftoff!')
        return

    print(n)
    countdown(n - 1)    # RECURSIVE CASE: call itself, smaller

countdown(3)

# Prints: 3, 2, 1, Liftoff!

```

Line by line:

Code	What it does
Base case	Without one, the function calls itself forever and crashes
<code>countdown(n-1)</code>	Each call handles a slightly smaller problem
In Min-Max	The base case is 'game over'; the recursive case is 'try a move, then evaluate'

Step 4 — Write Min-Max

Read this slowly. If the game has ended, return a score: +1 if X won, -1 if O won, 0 for a draw. Otherwise, try every empty cell. When it is X's turn (maximising), keep the highest score found. When it is O's turn (minimising), keep the lowest. The line that undoes the move is critical — without it the board fills up with imaginary moves.

```

def minimax(board, is_maximizing):
    result = check_winner(board)

    # BASE CASE: the game is over, so we know the score
    if result == 'X':
        return 1
    if result == 'O':

```

```

    return -1
if result == 'Draw':
    return 0

if is_maximizing:                # X's turn: wants the HIGHEST
    best_score = -float('inf')
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'        # try the move
            score = minimax(board, False)
            board[i] = ' '        # UNDO the move
            best_score = max(best_score, score)
    return best_score

else:                             # 0's turn: wants the LOWEST
    best_score = float('inf')
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax(board, True)
            board[i] = ' '
            best_score = min(best_score, score)
    return best_score

```

Line by line:

Code	What it does
<code>-float('inf')</code>	Negative infinity — any real score will beat it
<code>board[i] = 'X'</code>	Tentatively make a move
<code>minimax(board, False)</code>	Recurse, and now it is the OTHER player's turn
<code>board[i] = ' '</code>	UNDO. This is the most commonly forgotten line
<code>max vs min</code>	The single difference between the MAX player and the MIN player

Step 5 — Choose the best move

Min-Max tells you how good a position is. To actually play, we try each legal move, ask Min-Max to score the resulting position, and keep whichever move scored highest.

```

def best_move(board):
    best_score = -float('inf')
    move = None

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(board, False)
            board[i] = ' '

            if score > best_score:
                best_score = score
                move = i

    return move

```

Line by line:

Code	What it does
<code>for i in range(9)</code>	Consider every cell

Code	What it does
<code>if board[i] == ' '</code>	Only empty cells are legal moves
<code>minimax(board, False)</code>	After X moves it becomes O's turn, so <code>is_maximizing</code> is False
<code>if score > best_score</code>	Remember the best option found so far

Step 6 — Play a full game against the computer

A simple loop: you type a cell number, the computer replies with its optimal move. Try your hardest — you will find you can draw at best, never win.

```
def play_game():
    board = [' '] * 9

    while check_winner(board) is None:
        # Human plays O
        print_board(board)
        choice = int(input('Your move (0-8): '))

        if board[choice] != ' ':
            print('That cell is taken. Try again.')
            continue
        board[choice] = 'O'

        if check_winner(board) is not None:
            break

        # Computer plays X, optimally
        ai_move = best_move(board)
        board[ai_move] = 'X'
        print(f'Computer played {ai_move}')

    print_board(board)
    result = check_winner(board)
    print('Result:', 'Draw' if result == 'Draw' else f'{result} wins')
```

Line by line:

Code	What it does
<code>int(input(...))</code>	Reads what the user types and converts it to a number
<code>continue</code>	Skip back to the top of the loop without doing the rest
<code>is None</code>	The correct way to check for None in Python
<code>You cannot win</code>	The best you can ever achieve against perfect Min-Max play is a draw

The complete program (lab7.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab7.py – Unbeatable Tic-Tac-Toe with Min-Max

def print_board(board):
    for r in range(0, 9, 3):
        print(' ' + ' | '.join(board[r:r+3]))
        if r < 6:
            print(' -----')

def check_winner(board):
    lines = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a, b, c in lines:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]
    if ' ' not in board:
        return 'Draw'
    return None

def minimax(board, is_maximizing):
    result = check_winner(board)
    if result == 'X':
        return 1
    if result == 'O':
        return -1
    if result == 'Draw':
        return 0

    if is_maximizing:
        best_score = -float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax(board, False)
                board[i] = ' '
                best_score = max(best_score, score)
        return best_score
    else:
        best_score = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = minimax(board, True)
                board[i] = ' '
                best_score = min(best_score, score)
        return best_score

def best_move(board):
    best_score = -float('inf')
    move = None
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(board, False)
            board[i] = ' '
            if score > best_score:
                best_score = score
                move = i
```

```

    return move

# --- Demonstration: opponent took the centre, what does the AI do? ---
board = [' '] * 9
board[4] = 'O'

print('Opponent has taken the centre:')
print_board(board)
print()

ai_move = best_move(board)
board[ai_move] = 'X'

print(f'Min-Max chose cell {ai_move}:')
print_board(board)
print()
print('Theory says a corner (0, 2, 6 or 8) is the only correct reply.')

# --- Uncomment to play interactively ---
# def play_game():
#     board = [' '] * 9
#     while check_winner(board) is None:
#         print_board(board)
#         choice = int(input('Your move (0-8): '))
#         if board[choice] != ' ':
#             print('Taken, try again. '); continue
#         board[choice] = 'O'
#         if check_winner(board) is not None: break
#         board[best_move(board)] = 'X'
#     print_board(board)
#     print('Result:', check_winner(board))
# play_game()

```

HOW TO RUN IT

```
python lab7.py
```

WHAT YOU SHOULD SEE

Opponent has taken the centre:

```

| |
-----
| O |
-----
| |

```

Min-Max chose cell 0:

```

X | |
-----
| O |
-----
| |

```

Theory says a corner (0, 2, 6 or 8) is the only correct reply.

(The AI picks a corner, which is the game-theoretically correct response to an opponent taking the centre. It found this by exhaustively playing out every possible continuation of the game.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
RecursionError: maximum recursion depth exceeded	You forgot to undo the move, so the board never fills and the game never ends	Make sure <code>board[i] = ''</code> appears after every recursive call
The AI plays badly or randomly	<code>check_winner</code> is not detecting wins correctly	Test <code>check_winner</code> on a board with a known win before using it
AI always picks cell 0	minimax always returns the same score, so the first move always 'wins' the comparison	Verify the base cases return 1, -1 and 0 for the right players
The program takes several seconds on an empty board	This is normal — it evaluates over 500,000 positions	Lab 8 fixes this with alpha-beta pruning

TRY IT YOURSELF

- Uncomment `play_game()` and try genuinely hard to beat the computer. You cannot.
- Add a depth parameter so the AI prefers winning quickly (return `10 - depth` instead of 1).
- Count how many times minimax is called for an empty board — you will need this number in Lab 8.
- Make the AI play O instead of X by swapping which player maximises.

LAB 8: Enhance the Game with Alpha-Beta Pruning and Compare Efficiency

Relates to: Week 8 — Alpha-Beta Pruning

WHAT THIS LAB IS ABOUT

Min-Max in Lab 7 works, but it examines over half a million positions for an empty board — many of them pointlessly. Imagine you are choosing between two restaurants. You learn restaurant A scores 7. You start reading about restaurant B and immediately see a review scoring 3. You can stop reading about B right there: even if every other review is perfect, B's worst case is already worse than A's guaranteed 7. That is alpha-beta pruning. Alpha is the best score MAX can guarantee so far; beta is the best MIN can guarantee. When $\alpha \geq \beta$, the rest of that branch cannot possibly matter, so we skip it — and get the exact same answer for less work.

BEFORE YOU START, YOU NEED

- Completion of Lab 7 (you will extend that exact code)
- Understanding of how Min-Max recursion works

Building it step by step

Step 1 — Add a counter to plain Min-Max

Before we can prove pruning helps, we need a baseline. We add a global counter that ticks up every time the function is entered. This measures exactly how much work the algorithm does.

```
node_count = {'plain': 0, 'pruned': 0}

def minimax_plain(board, is_maximizing):
    node_count['plain'] += 1          # <-- count this visit

    result = check_winner(board)
    if result == 'X': return 1
    if result == 'O': return -1
    if result == 'Draw': return 0

    if is_maximizing:
        best = -float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                best = max(best, minimax_plain(board, False))
                board[i] = ' '
        return best
    else:
        best = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                best = min(best, minimax_plain(board, True))
                board[i] = ' '
        return best
```

Line by line:

Code	What it does
<code>node_count = {...}</code>	A dictionary holding both counters, so we can compare later
<code>node_count['plain'] += 1</code>	Increment on every entry into the function
Everything else	Identical to Lab 7

Step 2 — Understand alpha and beta

Alpha means: MAX has already found a way to guarantee at least this score. Beta means: MIN has already found a way to hold MAX to at most this score. They start at negative and positive infinity because at the beginning neither player has guaranteed anything.

```
# alpha = the best score MAX can already guarantee (starts at -infinity)
# beta  = the best score MIN can already guarantee (starts at +infinity)

# The pruning rule, in plain words:
#   If alpha >= beta, then the player about to move here would never
#   allow the game to reach this branch, because they already have a
#   better option elsewhere. So stop examining it.

alpha = -float('inf')
beta = float('inf')

# Example of pruning triggering:
alpha = 5      # MAX can already guarantee 5 somewhere else
beta = 3       # this branch can give MAX at most 3
print('Prune?', alpha >= beta)  # True – skip the rest of this branch
```

Line by line:

Code	What it does
alpha	MAX's guaranteed floor — only ever increases
beta	MIN's guaranteed ceiling — only ever decreases
alpha >= beta	The windows have crossed; nothing here can change the outcome

Step 3 — Write alpha-beta Min-Max

Compare this line by line with Step 1. Only three things are added: alpha and beta are passed in, they are updated after each child is evaluated, and a break statement escapes the loop when they cross. The returned value is identical to plain Min-Max — pruning never changes the answer.

```
def minimax_alphabeta(board, is_maximizing, alpha, beta):
    node_count['pruned'] += 1

    result = check_winner(board)
    if result == 'X': return 1
    if result == 'O': return -1
    if result == 'Draw': return 0

    if is_maximizing:
        best = -float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax_alphabeta(board, False, alpha, beta)
                board[i] = ' '

                best = max(best, score)
                alpha = max(alpha, best)      # MAX raises its floor

                if alpha >= beta:
                    break                    # PRUNE
        return best
```

```

else:
    best = float('inf')
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax_alphabeta(board, True, alpha, beta)
            board[i] = ' '

            best = min(best, score)
            beta = min(beta, best)          # MIN lowers its ceiling

            if alpha >= beta:
                break                      # PRUNE

    return best

```

Line by line:

Code	What it does
alpha, beta	Passed down through every recursive call
alpha = max(alpha, best)	Only MAX updates alpha
beta = min(beta, best)	Only MIN updates beta
break	Abandons the remaining moves in this branch — this is the pruning
Same return value	Pruning skips work, it never changes the result

Step 4 — Run both and compare

We reset the counters, run each version on an identical empty board, and compute the percentage saved.

```

node_count['plain'] = 0
node_count['pruned'] = 0

empty_board = [' '] * 9

score_plain = minimax_plain(list(empty_board), True)
score_pruned = minimax_alphabeta(list(empty_board), True, -float('inf'), float('inf'))

print('Result from plain Min-Max: ', score_plain)
print('Result from alpha-beta:    ', score_pruned)
print('Same answer?', score_plain == score_pruned)
print()
print('Positions examined (plain):    ', node_count['plain'])
print('Positions examined (alpha-beta):', node_count['pruned'])

reduction = 100 * (1 - node_count['pruned'] / node_count['plain'])
print(f'Reduction: {reduction:.1f}%')

```

Line by line:

Code	What it does
list(empty_board)	Passes a fresh copy so the two runs cannot interfere
score_plain == score_pruned	Should print True — proof that pruning is safe
1 - pruned/plain	The fraction of work avoided

The complete program (lab8.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab8.py – Alpha-beta pruning versus plain Min-Max

node_count = {'plain': 0, 'pruned': 0}

def check_winner(board):
    lines = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a, b, c in lines:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]
    if ' ' not in board:
        return 'Draw'
    return None

def minimax_plain(board, is_maximizing):
    node_count['plain'] += 1
    result = check_winner(board)
    if result == 'X': return 1
    if result == 'O': return -1
    if result == 'Draw': return 0

    if is_maximizing:
        best = -float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                best = max(best, minimax_plain(board, False))
                board[i] = ' '
        return best
    else:
        best = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                best = min(best, minimax_plain(board, True))
                board[i] = ' '
        return best

def minimax_alphabeta(board, is_maximizing, alpha, beta):
    node_count['pruned'] += 1
    result = check_winner(board)
    if result == 'X': return 1
    if result == 'O': return -1
    if result == 'Draw': return 0

    if is_maximizing:
        best = -float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax_alphabeta(board, False, alpha, beta)
                board[i] = ' '
                best = max(best, score)
                alpha = max(alpha, best)
                if alpha >= beta:
                    break
        return best
```

```

else:
    best = float('inf')
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax_alphabeta(board, True, alpha, beta)
            board[i] = ' '
            best = min(best, score)
            beta = min(beta, best)
            if alpha >= beta:
                break
    return best

empty_board = [' '] * 9

score_plain = minimax_plain(list(empty_board), True)
score_pruned = minimax_alphabeta(list(empty_board), True,
                                -float('inf'), float('inf'))

print('=== Correctness check ===')
print('Plain Min-Max result:', score_plain)
print('Alpha-beta result:   ', score_pruned)
print('Identical?', score_plain == score_pruned)
print()
print('=== Efficiency comparison ===')
print('Positions examined (plain):      ', node_count['plain'])
print('Positions examined (alpha-beta): ', node_count['pruned'])

reduction = 100 * (1 - node_count['pruned'] / node_count['plain'])
print(f'Work avoided: {reduction:.1f}%')
print()
print('Both return 0, meaning perfect play from both sides is a draw.')

```

HOW TO RUN IT

```
python lab8.py
```

WHAT YOU SHOULD SEE

```

=== Correctness check ===
Plain Min-Max result: 0
Alpha-beta result:    0
Identical? True

=== Efficiency comparison ===
Positions examined (plain):      549946
Positions examined (alpha-beta): 18297
Work avoided: 96.7%

Both return 0, meaning perfect play from both sides is a draw.

(The headline result: alpha-beta examined roughly 30 times fewer positions
while producing the exact same answer. On a game like chess, this is the
difference between searching 4 moves ahead and 8 moves ahead.)

```

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
Alpha-beta gives a different result from plain Min-Max	alpha and beta were updated by the wrong player	MAX updates alpha only; MIN updates beta only. Never swap them
No reduction at all (identical)	The break statement is missing or	Check that break is inside the for loop, after the

Error message / symptom	Why it happened	How to fix it
counts)	unreachable	alpha/beta update
UnboundLocalError: node_count	Python treats assignment to a name as local unless it is a mutable object	Use a dictionary as shown, or declare global node_count
RecursionError	Same cause as Lab 7 — a missing undo	Confirm board[i] = '' runs after each recursive call

TRY IT YOURSELF

- Reorder the loop to try the centre and corners first (for i in [4,0,2,6,8,1,3,5,7]). Does the reduction improve? Good move ordering makes pruning far more effective.
- Reverse the move order (range(8,-1,-1)) and see whether pruning becomes worse.
- Print a message each time a branch is pruned, to see where and how often it happens.
- Time both versions with time.perf_counter() and report the speedup in milliseconds as well as node counts.

LAB 9: Build a Small Rule-Based Expert System

Relates to: Week 2 — Knowledge Representation and Inference

WHAT THIS LAB IS ABOUT

An expert system stores a human expert's knowledge as IF-THEN rules and then chains those rules together automatically. If we know 'fever AND cough AND sore throat means flu', and separately 'flu AND chest pain means possible pneumonia', then a patient with all four symptoms should trigger both rules in sequence — the conclusion of the first becomes an input to the second. That chaining is the whole idea, and it is called forward chaining. This was how 1980s medical AI worked, and the technique is still used today in insurance, diagnostics, and compliance checking.

BEFORE YOU START, YOU NEED

- Completion of Labs 1–2
- Understanding of Python sets (explained in Step 1)
- No external libraries

Building it step by step

Step 1 — Learn Python sets in two minutes

A set is an unordered collection with no duplicates. The critical operation for this lab is `issubset`: checking whether every item in one set also appears in another. That is exactly how we test whether a rule's conditions are all satisfied.

```
symptoms = {'fever', 'cough', 'sore_throat'}

print('fever' in symptoms)           # True

required = {'fever', 'cough'}
print(required.issubset(symptoms))   # True — all present

required2 = {'fever', 'rash'}
print(required2.issubset(symptoms))  # False — rash missing

symptoms.add('headache')             # sets can grow
print(symptoms)
```

Line by line:

Code	What it does
<code>{'a', 'b'}</code>	Curly braces with no colons creates a set
<code>in</code>	Fast membership test
<code>issubset</code>	True only if EVERY item of the smaller set is in the bigger one
<code>add</code>	Inserts an item; duplicates are silently ignored

Step 2 — Write the knowledge base

Each rule is a pair: the set of facts that must be true, and the single fact we may then conclude. Read the third rule carefully — its condition includes 'flu', which is itself a conclusion of rule one. That is what makes chaining possible.

```
rules = [
    ({'fever', 'cough', 'sore_throat'}, 'flu'),
    ({'fever', 'rash', 'joint_pain'}, 'dengue'),
    ({'flu', 'chest_pain'}, 'possible_pneumonia'),
    ({'sneezing', 'runny_nose', 'itchy_eyes'}, 'allergy'),
    ({'possible_pneumonia'}, 'refer_to_specialist'),
]
```

```
for conditions, conclusion in rules:
    print('IF', conditions, 'THEN', conclusion)
```

Line by line:

Code	What it does
<code>{...}, 'flu'</code>	A tuple: the condition set first, the conclusion second
<code>'flu', 'chest_pain'</code>	Depends on 'flu', which rule 1 produces — enabling a chain
<code>'possible_pneumonia'</code>	A single-condition rule; the chain can keep going

Step 3 — Write the inference engine

The engine repeatedly sweeps through every rule. If a rule's conditions are satisfied and we do not already know its conclusion, we add that conclusion to our facts and note that something changed. We keep sweeping until a full pass produces nothing new — that is called reaching a fixed point.

```
def forward_chain(initial_facts, rules):
    facts = set(initial_facts)
    fired = []

    changed = True
    while changed:
        changed = False

        for conditions, conclusion in rules:
            all_conditions_met = conditions.issubset(facts)
            already_known = conclusion in facts

            if all_conditions_met and not already_known:
                facts.add(conclusion)
                fired.append((conditions, conclusion))
                changed = True          # something new -> sweep again

    return facts, fired
```

Line by line:

Code	What it does
<code>set(initial_facts)</code>	Makes a copy, so we never modify the caller's data
<code>while changed:</code>	Keep sweeping as long as the last pass learned something
<code>changed = False</code>	Reset at the start of each sweep
<code>not already_known</code>	Prevents re-firing the same rule forever
<code>fired</code>	Records the chain of reasoning, so we can explain the conclusion

Step 4 — Run a diagnosis and explain it

Being able to show WHY the system reached a conclusion is what made expert systems trusted in medicine. Our fired list gives us exactly that, for free.

```
patient = {'fever', 'cough', 'sore_throat', 'chest_pain'}

final_facts, fired = forward_chain(patient, rules)

print('Reported symptoms:', patient)
print()
```

```
print('Chain of reasoning:')
for step, (conditions, conclusion) in enumerate(fired, start=1):
    print(f' {step}. Because {sorted(conditions)}')
    print(f'      we conclude: {conclusion}')

print()
new_facts = final_facts - patient
print('Everything inferred:', sorted(new_facts))
```

Line by line:

Code	What it does
<code>enumerate(fired, start=1)</code>	Numbers the steps starting from 1 instead of 0
<code>sorted(conditions)</code>	Sets have no order, so we sort for tidy printing
<code>final_facts - patient</code>	Set difference: what was DEDUCED, excluding what was given

The complete program (lab9.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab9.py – A rule-based expert system with forward chaining

rules = [
    ({'fever', 'cough', 'sore_throat'},      'flu'),
    ({'fever', 'rash', 'joint_pain'},        'dengue'),
    ({'flu', 'chest_pain'},                  'possible_pneumonia'),
    ({'sneezing', 'runny_nose', 'itchy_eyes'}, 'allergy'),
    ({'possible_pneumonia'},                 'refer_to_specialist'),
]

def forward_chain(initial_facts, rules):
    facts = set(initial_facts)
    fired = []
    changed = True

    while changed:
        changed = False
        for conditions, conclusion in rules:
            if conditions.issubset(facts) and conclusion not in facts:
                facts.add(conclusion)
                fired.append((conditions, conclusion))
                changed = True

    return facts, fired

def diagnose(patient_symptoms, label):
    print('=' * 55)
    print('PATIENT:', label)
    print('Reported symptoms:', sorted(patient_symptoms))
    print()

    final_facts, fired = forward_chain(patient_symptoms, rules)

    if not fired:
        print('No conclusion could be drawn from these symptoms.')
    else:
        print('Chain of reasoning:')
        for step, (conditions, conclusion) in enumerate(fired, start=1):
            print(f'  {step}. Given {sorted(conditions)}')
            print(f'      conclude -> {conclusion}')
        print()
        deduced = sorted(final_facts - set(patient_symptoms))
        print('Everything deduced:', deduced)
    print()

# Case 1: chains through three rules
diagnose({'fever', 'cough', 'sore_throat', 'chest_pain'}, 'A')

# Case 2: a single rule fires
diagnose({'sneezing', 'runny_nose', 'itchy_eyes'}, 'B')

# Case 3: nothing matches
diagnose({'fever', 'headache'}, 'C')
```

HOW TO RUN IT

```
python lab9.py
```

WHAT YOU SHOULD SEE

```
=====
PATIENT: A
Reported symptoms: ['chest_pain', 'cough', 'fever', 'sore_throat']

Chain of reasoning:
  1. Given ['cough', 'fever', 'sore_throat']
     conclude -> flu
  2. Given ['chest_pain', 'flu']
     conclude -> possible_pneumonia
  3. Given ['possible_pneumonia']
     conclude -> refer_to_specialist

Everything deduced: ['flu', 'possible_pneumonia', 'refer_to_specialist']

=====
PATIENT: B
Reported symptoms: ['itchy_eyes', 'runny_nose', 'sneezing']

Chain of reasoning:
  1. Given ['itchy_eyes', 'runny_nose', 'sneezing']
     conclude -> allergy

Everything deduced: ['allergy']

=====
PATIENT: C
Reported symptoms: ['fever', 'headache']

No conclusion could be drawn from these symptoms.

(Patient A shows the three-step chain: symptoms -> flu -> pneumonia ->
referral. No single rule knew about referrals from raw symptoms; the
system worked it out by chaining.)
```

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
Program loops forever	changed is never set back to False at the start of each sweep	Set changed = False as the first line inside the while loop
AttributeError: 'list' object has no attribute 'issubset'	Conditions were written with square brackets instead of curly braces	Use {'fever', 'cough'} not ['fever', 'cough']
Only the first rule ever fires	The chaining loop is missing — you looped through rules only once	The for loop must sit inside a while loop
Conclusions appear multiple times	The 'conclusion not in facts' check is missing	Always check before adding, or the same rule refires endlessly

TRY IT YOURSELF

- Add rules for a different domain entirely, such as diagnosing why a car will not start.
- Add a rule whose conclusion feeds a fourth rule, making a longer chain.
- Add confidence scores (e.g., flu at 0.8) and multiply them along the chain.
- Write backward chaining instead: start from a hypothesis such as 'flu' and work out which symptoms would confirm it.

LAB 10: Build a Decision Tree Classifier From Scratch

Relates to: Week 10 — Supervised Learning, Entropy, Information Gain

WHAT THIS LAB IS ABOUT

A decision tree is a flowchart of questions. To decide whether to play tennis, you might first ask 'what is the outlook?' then, depending on the answer, ask about humidity or wind. The clever part is choosing which question to ask first. The answer comes from information theory: ask the question that reduces uncertainty the most. Uncertainty is measured by entropy, and the reduction achieved by a question is called information gain. This lab implements both formulas by hand, then uses them to grow a tree automatically.

BEFORE YOU START, YOU NEED

- Completion of Labs 1–2
- Understanding of Python dictionaries and list comprehensions
- Basic idea of logarithms (log base 2)

Building it step by step

Step 1 — Prepare the classic dataset

Each row is one day, described by four features, with a label saying whether tennis was played. This is the standard teaching dataset for decision trees.

```
data = [
    {'Outlook': 'Sunny', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'No'},
    {'Outlook': 'Sunny', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Strong', 'Play': 'No'},
    {'Outlook': 'Overcast', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Mild', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong', 'Play': 'No'},
    {'Outlook': 'Overcast', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong', 'Play': 'Yes'},
    {'Outlook': 'Sunny', 'Temp': 'Mild', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'No'},
]

print('Rows:', len(data))
print('Played tennis:', sum(1 for r in data if r['Play'] == 'Yes'))
print('Did not play: ', sum(1 for r in data if r['Play'] == 'No'))
```

Line by line:

Code	What it does
Each dictionary	One day's observations plus the outcome
'Play'	The label we are trying to predict
sum(1 for ... if ...)	Counts how many rows satisfy a condition

Step 2 — Understand and code entropy

Entropy measures how mixed up a group is. If every row has the same label, entropy is 0 — no uncertainty. If it is a perfect 50/50 split, entropy is 1 — maximum uncertainty. The formula sums, over each class, minus the proportion times the log base 2 of the proportion.

```
import math
from collections import Counter

def entropy(rows):
    counts = Counter(row['Play'] for row in rows)
    total = len(rows)
```

```

result = 0
for label_count in counts.values():
    proportion = label_count / total
    result -= proportion * math.log2(proportion)

return result

# Try it on some simple cases:
all_yes = [{'Play':'Yes'}, {'Play':'Yes'}, {'Play':'Yes'}]
print('All the same:', entropy(all_yes))          # 0.0 — no uncertainty

half = [{'Play':'Yes'}, {'Play':'No'}]
print('Even split: ', entropy(half))              # 1.0 — max uncertainty

print('Our dataset: ', entropy(data))             # 1.0 (4 Yes, 4 No)

```

Line by line:

Code	What it does
Counter(...)	Counts how many of each label appear
proportion	The fraction of rows with this label
math.log2	Logarithm base 2, which is why entropy is measured in bits
result -=	The formula has a leading minus sign
Entropy 0	Perfectly pure — the goal of every split

Step 3 — Code information gain

Information gain answers: if I split the data on this feature, how much does the average uncertainty drop? We compute the entropy of each resulting group, weight each by how many rows it contains, and subtract that weighted total from the original entropy.

```

def information_gain(rows, attribute):
    starting_entropy = entropy(rows)

    values = set(row[attribute] for row in rows)
    weighted_entropy = 0

    for value in values:
        subset = [row for row in rows if row[attribute] == value]
        weight = len(subset) / len(rows)
        weighted_entropy += weight * entropy(subset)

    return starting_entropy - weighted_entropy

# Which question is most informative?
for attribute in ['Outlook', 'Temp', 'Humidity', 'Wind']:
    gain = information_gain(data, attribute)
    print(f'{attribute:<10} gain = {gain:.4f}')

```

Line by line:

Code	What it does
set(row[attribute] ...)	All the distinct values this feature can take
subset	Only the rows having this particular value
weight	Bigger groups matter more in the average

Code	What it does
starting - weighted	The reduction in uncertainty: the gain
Highest gain wins	That feature becomes the next question in the tree

Step 4 — Grow the tree recursively

This is the ID3 algorithm. Three stopping conditions come first: if all labels match, return that label; if we have run out of questions, return the majority answer. Otherwise pick the highest-gain feature, split on it, and recurse on each piece.

```
def build_tree(rows, attributes):
    labels = [row['Play'] for row in rows]

    # STOP 1: every row has the same label – a pure leaf
    if labels.count(labels[0]) == len(labels):
        return labels[0]

    # STOP 2: no questions left – go with the majority
    if not attributes:
        return Counter(labels).most_common(1)[0][0]

    # Pick the most informative question
    gains = {attr: information_gain(rows, attr) for attr in attributes}
    best_attribute = max(gains, key=gains.get)

    tree = {best_attribute: {}}
    remaining = [a for a in attributes if a != best_attribute]

    for value in set(row[best_attribute] for row in rows):
        subset = [row for row in rows if row[best_attribute] == value]
        tree[best_attribute][value] = build_tree(subset, remaining)

    return tree
```

Line by line:

Code	What it does
labels.count(labels[0]) == len(labels)	Every label equals the first one, so the node is pure
most_common(1)[0][0]	The single most frequent label
{attr: ... for attr in ...}	A dictionary comprehension building feature-to-gain pairs
max(gains, key=gains.get)	The feature name with the biggest gain value
build_tree(subset, remaining)	Recursion: solve the same problem on a smaller piece

Step 5 — Use the tree to classify

Walk down the tree. At each node, look at which feature it asks about, read that feature from the example, and follow the matching branch. When you reach something that is not a dictionary, that is a leaf: the answer.

```
def classify(tree, example):
    # A leaf is a plain string, not a dictionary
    if not isinstance(tree, dict):
        return tree

    attribute = next(iter(tree))    # which feature this node asks about
    value = example[attribute]      # the example's answer
```

```

    if value not in tree[attribute]:
        return 'Unknown'                # value never seen in training

    return classify(tree[attribute][value], example)

def print_tree(tree, indent=''):
    if not isinstance(tree, dict):
        print(indent + '-> ' + tree)
        return
    attribute = next(iter(tree))
    for value, subtree in tree[attribute].items():
        print(f'{indent}{attribute} = {value}?')
        print_tree(subtree, indent + '    ')

```

Line by line:

Code	What it does
<code>isinstance(tree, dict)</code>	Checks whether we are at a branch or a leaf
<code>next(iter(tree))</code>	Gets the single key from a one-key dictionary
<code>classify(subtree, example)</code>	Recursion again, descending one level
<code>'Unknown'</code>	Graceful handling of unseen values

The complete program (lab10.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab10.py – A decision tree built from scratch (ID3)
import math
from collections import Counter

data = [
    {'Outlook': 'Sunny', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'No'},
    {'Outlook': 'Sunny', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Strong', 'Play': 'No'},
    {'Outlook': 'Overcast', 'Temp': 'Hot', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Mild', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Weak', 'Play': 'Yes'},
    {'Outlook': 'Rain', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong', 'Play': 'No'},
    {'Outlook': 'Overcast', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong', 'Play': 'Yes'},
    {'Outlook': 'Sunny', 'Temp': 'Mild', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'No'},
]

def entropy(rows):
    counts = Counter(row['Play'] for row in rows)
    total = len(rows)
    result = 0
    for label_count in counts.values():
        proportion = label_count / total
        result -= proportion * math.log2(proportion)
    return result

def information_gain(rows, attribute):
    starting_entropy = entropy(rows)
    weighted_entropy = 0
    for value in set(row[attribute] for row in rows):
        subset = [row for row in rows if row[attribute] == value]
        weighted_entropy += (len(subset)/len(rows)) * entropy(subset)
    return starting_entropy - weighted_entropy

def build_tree(rows, attributes):
    labels = [row['Play'] for row in rows]
    if labels.count(labels[0]) == len(labels):
        return labels[0]
    if not attributes:
        return Counter(labels).most_common(1)[0][0]
    gains = {attr: information_gain(rows, attr) for attr in attributes}
    best_attribute = max(gains, key=gains.get)
    tree = {best_attribute: {}}
    remaining = [a for a in attributes if a != best_attribute]
    for value in set(row[best_attribute] for row in rows):
        subset = [row for row in rows if row[best_attribute] == value]
        tree[best_attribute][value] = build_tree(subset, remaining)
    return tree

def classify(tree, example):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    value = example[attribute]
    if value not in tree[attribute]:
        return 'Unknown'
    return classify(tree[attribute][value], example)
```

```
def print_tree(tree, indent=''):
    if not isinstance(tree, dict):
        print(indent + '-> ' + tree)
        return
    attribute = next(iter(tree))
    for value, subtree in tree[attribute].items():
        print(f'{indent}{attribute} = {value}?')
        print_tree(subtree, indent + '    ')

print('Starting entropy:', round(entropy(data), 4))
print()
print('Information gain per feature:')
for attribute in ['Outlook', 'Temp', 'Humidity', 'Wind']:
    print(f' {attribute}:<10> {information_gain(data, attribute):.4f}')
print()

tree = build_tree(data, ['Outlook', 'Temp', 'Humidity', 'Wind'])
print('Learned tree:')
print_tree(tree)
print()

new_day = {'Outlook': 'Sunny', 'Temp': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong'}
print('New day:', new_day)
print('Prediction:', classify(tree, new_day))
```

HOW TO RUN IT

```
python lab10.py
```

WHAT YOU SHOULD SEE

Starting entropy: 1.0

Information gain per feature:

```
Outlook    0.6556
Temp       0.2045
Humidity   0.3444
Wind       0.1616
```

Learned tree:

```
Outlook = Rain?
  Wind = Weak?
    -> Yes
  Wind = Strong?
    -> No
Outlook = Sunny?
  -> No
Outlook = Overcast?
  -> Yes
```

New day: {'Outlook': 'Sunny', 'Temp': 'Cool', ...}

Prediction: No

(Outlook has by far the highest information gain, so it becomes the root question – exactly matching the by-hand calculation from lecture. Sunny and Overcast are immediately pure, so they need no further questions.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
ValueError: math domain error	You tried to take log2 of zero	A class with zero count should never enter the

Error message / symptom	Why it happened	How to fix it
		loop; use Counter, which only stores classes that appear
IndexError: list index out of range	build_tree was called with an empty subset	Add a guard: if not rows, return the majority label of the parent
RecursionError	An attribute was never removed from the list, so it splits forever	Confirm remaining excludes best_attribute
Tree always predicts the same class	information_gain returns 0 for everything, likely an entropy bug	Test entropy on all-same and 50/50 inputs first; they must give 0.0 and 1.0

TRY IT YOURSELF

- Add more rows to the dataset and see whether the root question changes.
- Implement the Gini index as an alternative to entropy and compare the trees produced.
- Add a maximum depth limit to stop the tree growing too specific (this is pre-pruning).
- Print the information gain at every node, not just the root.

LAB 11: Implement a Naïve Bayes Classifier for Spam Detection

Relates to: Weeks 11–12 — Bayes' Theorem, Naïve Bayes

WHAT THIS LAB IS ABOUT

Bayes' Theorem lets you update a belief when new evidence arrives. For spam filtering the belief is 'this email is spam', and the evidence is the words it contains. The word 'free' is common in spam and rare in real mail, so seeing it should push your belief towards spam. Naïve Bayes multiplies together the evidence from every word. It is called naive because it pretends words are independent of each other, which is obviously false — yet it works remarkably well in practice, and it is fast enough to filter millions of emails.

BEFORE YOU START, YOU NEED

- Completion of Labs 1–2
- Understanding of Python dictionaries and loops
- Familiarity with Bayes' Theorem from Week 11 lectures

Building it step by step

Step 1 — Gather training data and count words

We need to know how often each word appears in spam versus real mail. Counter and defaultdict make this bookkeeping painless. defaultdict(Counter) means: if I ask for a key that does not exist, quietly create an empty Counter for it.

```
from collections import Counter, defaultdict

training_data = [
    ('win free money now',      'spam'),
    ('claim your free prize today', 'spam'),
    ('limited offer act now',    'spam'),
    ('meeting scheduled for tomorrow', 'ham'),
    ('can we discuss the project',   'ham'),
    ('lunch at noon works for me',   'ham'),
]

def tokenize(text):
    return text.lower().split()

word_counts = defaultdict(Counter) # word_counts['spam']['free'] -> 2
total_words = Counter()             # how many words in each class
doc_counts = Counter()              # how many messages in each class
vocabulary = set()

for text, label in training_data:
    doc_counts[label] += 1
    for word in tokenize(text):
        word_counts[label][word] += 1
        total_words[label] += 1
        vocabulary.add(word)

print('Spam messages:', doc_counts['spam'])
print("'free' appears in spam:", word_counts['spam']['free'], 'times')
print("'free' appears in ham: ", word_counts['ham']['free'], 'times')
print('Vocabulary size:', len(vocabulary))
```

Line by line:

Code	What it does
<code>text.lower().split()</code>	Lowercases then splits on spaces into a list of words

Code	What it does
<code>defaultdict(Counter)</code>	Auto-creates an empty Counter for any new class
<code>vocabulary</code>	Every distinct word seen anywhere — needed for smoothing

Step 2 — Compute the prior probability

Before reading a single word, what fraction of messages are spam? That is the prior. If half your mail is spam, the prior is 0.5. This is the starting belief that the evidence will then adjust.

```
total_docs = sum(doc_counts.values())

def prior(label):
    return doc_counts[label] / total_docs

print('P(spam) =', prior('spam'))      # 3/6 = 0.5
print('P(ham) =', prior('ham'))        # 3/6 = 0.5
```

Line by line:

Code	What it does
<code>doc_counts[label] / total_docs</code>	The fraction of all messages belonging to this class
Prior	Your belief before looking at any evidence

Step 3 — Compute word likelihoods with smoothing

$P(\text{word given class})$ is how likely that word is within that class. There is a trap: if a word never appeared in spam during training, its probability is zero, and multiplying by zero destroys the whole calculation. Laplace smoothing fixes this by pretending every word was seen one extra time.

```
vocab_size = len(vocabulary)

def likelihood(word, label):
    count = word_counts[label][word]
    # Add-1 (Laplace) smoothing: +1 on top, +vocab_size on the bottom
    return (count + 1) / (total_words[label] + vocab_size)

print("P('free' | spam) =", round(likelihood('free', 'spam'), 4))
print("P('free' | ham) =", round(likelihood('free', 'ham'), 4))
print()
print('Without smoothing, P(free|ham) would be 0, which would')
print('wipe out the entire ham score for any message containing it.')
```

Line by line:

Code	What it does
<code>count + 1</code>	The Laplace correction — pretend we saw it once more
<code>+ vocab_size</code>	Keeps all probabilities adding to 1
Never zero	Which is the whole point of smoothing

Step 4 — Classify, using logarithms

Multiplying many small probabilities together produces numbers so tiny the computer rounds them to zero. The standard fix is to add logarithms instead of multiplying probabilities. Adding logs is mathematically equivalent to multiplying, and whichever class has the highest log score also has the highest probability.

```
import math

def classify(text):
    words = tokenize(text)
    scores = {}

    for label in doc_counts:
        score = math.log(prior(label))
        for word in words:
            score += math.log(likelihood(word, label))
        scores[label] = score

    best = max(scores, key=scores.get)
    return best, scores

prediction, scores = classify('win a free prize')
print('Prediction:', prediction)
print('Scores:', {k: round(v, 3) for k, v in scores.items()})
```

Line by line:

Code	What it does
<code>math.log(...)</code>	Natural logarithm
<code>score += math.log(...)</code>	Adding logs replaces multiplying probabilities
Negative numbers	Normal — logs of values below 1 are always negative
<code>max(scores, key=scores.get)</code>	The class with the highest (least negative) score

Step 5 — Convert scores back to readable percentages

Log scores are hard to interpret. We can convert them back to probabilities that sum to 100%. The trick of subtracting the maximum first prevents numerical overflow.

```
def to_percentages(scores):
    biggest = max(scores.values())
    exp_scores = {k: math.exp(v - biggest) for k, v in scores.items()}
    total = sum(exp_scores.values())
    return {k: 100 * v / total for k, v in exp_scores.items()}

prediction, scores = classify('win a free prize')
percentages = to_percentages(scores)

for label, pct in percentages.items():
    print(f' {label}: {pct:.1f}%')
```

Line by line:

Code	What it does
<code>v - biggest</code>	Shifts all values down so <code>exp()</code> never overflows
<code>math.exp</code>	Undoes the logarithm
<code>100 * v / total</code>	Normalises so the percentages add up to 100

The complete program (lab11.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab11.py – Naïve Bayes spam detection from scratch
import math
from collections import Counter, defaultdict

training_data = [
    ('win free money now', 'spam'),
    ('claim your free prize today', 'spam'),
    ('limited offer act now', 'spam'),
    ('win big prize money', 'spam'),
    ('meeting scheduled for tomorrow', 'ham'),
    ('can we discuss the project', 'ham'),
    ('lunch at noon works for me', 'ham'),
    ('please review the attached file', 'ham'),
]

def tokenize(text):
    return text.lower().split()

word_counts = defaultdict(Counter)
total_words = Counter()
doc_counts = Counter()
vocabulary = set()

for text, label in training_data:
    doc_counts[label] += 1
    for word in tokenize(text):
        word_counts[label][word] += 1
        total_words[label] += 1
        vocabulary.add(word)

total_docs = sum(doc_counts.values())
vocab_size = len(vocabulary)

def prior(label):
    return doc_counts[label] / total_docs

def likelihood(word, label):
    return (word_counts[label][word] + 1) / (total_words[label] + vocab_size)

def classify(text):
    words = tokenize(text)
    scores = {}
    for label in doc_counts:
        score = math.log(prior(label))
        for word in words:
            score += math.log(likelihood(word, label))
        scores[label] = score
    return max(scores, key=scores.get), scores

def to_percentages(scores):
    biggest = max(scores.values())
    exp_scores = {k: math.exp(v - biggest) for k, v in scores.items()}
    total = sum(exp_scores.values())
    return {k: 100 * v / total for k, v in exp_scores.items()}

print('=== Training summary ===')
```

```

print('Messages:', total_docs, '| Vocabulary:', vocab_size)
print('P(spam) =', prior('spam'), '| P(ham) =', prior('ham'))
print()

test_messages = [
    'win a free prize now',
    'can we schedule the meeting tomorrow',
    'free lunch at the office',
]

for message in test_messages:
    prediction, scores = classify(message)
    percentages = to_percentages(scores)
    print(f'Message: "{message}"')
    print(f' Verdict: {prediction.upper()}')
    for label in sorted(percentages):
        print(f'    {label}: {percentages[label]:.1f}%')
    print()

```

HOW TO RUN IT

```
python lab11.py
```

WHAT YOU SHOULD SEE

```

=== Training summary ===
Messages: 8 | Vocabulary: 29
P(spam) = 0.5 | P(ham) = 0.5

Message: "win a free prize now"
Verdict: SPAM
  ham: 4.2%
 spam: 95.8%

Message: "can we schedule the meeting tomorrow"
Verdict: HAM
  ham: 97.1%
 spam: 2.9%

Message: "free lunch at the office"
Verdict: HAM
  ham: 61.3%
 spam: 38.7%

(The third message is the interesting one: 'free' pushes towards spam,
but 'lunch' and 'at' push towards ham, and ham narrowly wins. This is
the evidence-combining behaviour of Bayes' Theorem in action.)

```

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
ValueError: math domain error	You took log of zero — smoothing is missing or wrong	Verify likelihood adds 1 to the numerator and vocab_size to the denominator
Every message is classified the same way	The priors are lopsided, or one class has far more training data	Balance your training set, or check the likelihood denominators
KeyError on a word	You used a plain dict instead of Counter or defaultdict	Counter returns 0 for unseen keys instead of raising an error
All scores come out as -inf	log(0) occurred somewhere	Same fix as the first error — apply smoothing everywhere

TRY IT YOURSELF

- Remove smoothing (drop the +1 and +vocab_size) and find a message that breaks the classifier.
- Add far more spam examples than ham and watch the prior skew the results.
- Strip punctuation in tokenize() so 'free!' and 'free' count as the same word.
- Split your data into a training half and a testing half, and measure real accuracy.

LAB 12: Compare Naïve Bayes and SVM Using scikit-learn

Relates to: Week 12 — Naïve Bayes and Support Vector Machines

WHAT THIS LAB IS ABOUT

In Lab 11 you built Naïve Bayes by hand, which is the best way to understand it. In real work nobody does that — they use scikit-learn, which provides dozens of algorithms behind one consistent interface: create the model, call fit to train it, call predict to use it. This lab introduces that workflow and uses it to compare Naïve Bayes against a Support Vector Machine. An SVM works completely differently: instead of counting probabilities, it draws the boundary line that leaves the widest possible gap between the classes.

BEFORE YOU START, YOU NEED

- Completion of Lab 11
- scikit-learn installed (pip install scikit-learn)
- No dataset file needed — scikit-learn includes one

Building it step by step

Step 1 — Load a built-in dataset

The Iris dataset records four measurements of 150 flowers across three species. X holds the measurements (the inputs); y holds the species number (the answer). This X and y naming convention is universal in machine learning.

```
from sklearn.datasets import load_iris

data = load_iris()
X = data.data      # the measurements: 150 rows, 4 columns
y = data.target    # the answers: 150 values, each 0, 1 or 2

print('Shape of X:', X.shape)          # (150, 4)
print('Feature names:', data.feature_names)
print('Species names:', data.target_names)
print()
print('First flower measurements:', X[0])
print('First flower species:', y[0], '=', data.target_names[y[0]])
```

Line by line:

Code	What it does
X	The input features — capital X by convention, because it is a matrix
y	The target labels — lowercase y, because it is a single column
X.shape	Returns (rows, columns)
data.target_names	Turns the numbers 0, 1, 2 back into readable species names

Step 2 — Split into training and testing sets

You must never judge a model on data it has already seen — that is like giving a student the exam answers in advance. We hold back 20% of the flowers, train on the other 80%, then test on the held-back portion. stratify=y ensures each species appears in the same proportion in both halves.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # hold back 20% for testing
    random_state=42,    # makes the split repeatable
```

```

    stratify=y          # keep species balanced in both halves
)

print('Training flowers:', len(X_train))    # 120
print('Testing flowers: ', len(X_test))     # 30

```

Line by line:

Code	What it does
test_size=0.2	20% goes to testing, 80% to training
random_state=42	Fixes the randomness so your results match this handbook
stratify=y	Prevents an unlucky split that puts all of one species in testing
Four return values	Training inputs, testing inputs, training answers, testing answers

Step 3 — Train Naïve Bayes in three lines

Compare this with the fifty-odd lines you wrote in Lab 11. GaussianNB is the version of Naïve Bayes for continuous numbers rather than word counts. The pattern — create, fit, predict — is identical for every scikit-learn model.

```

from sklearn.naive_bayes import GaussianNB
import time

nb_model = GaussianNB()          # 1. create

start = time.perf_counter()
nb_model.fit(X_train, y_train)    # 2. train
nb_time = time.perf_counter() - start

nb_predictions = nb_model.predict(X_test) # 3. predict

print('Trained in', round(nb_time, 5), 'seconds')
print('First 10 predictions:', nb_predictions[:10])
print('First 10 true answers:', y_test[:10])

```

Line by line:

Code	What it does
.fit(X_train, y_train)	Learns from the training data
.predict(X_test)	Makes predictions on data it has never seen
GaussianNB	Naïve Bayes assuming each feature follows a bell curve
[:10]	Slicing — the first ten items

Step 4 — Train an SVM the same way

The only line that changes is which model you create. An SVM finds the boundary with the widest margin between classes. The rbf kernel lets it draw curved boundaries, not just straight lines.

```

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=1.0)

start = time.perf_counter()
svm_model.fit(X_train, y_train)
svm_time = time.perf_counter() - start

svm_predictions = svm_model.predict(X_test)

```

```
print('Trained in', round(svm_time, 5), 'seconds')
print('Support vectors used:', svm_model.n_support_)
```

Line by line:

Code	What it does
kernel='rbf'	Allows curved decision boundaries
C=1.0	Controls the tradeoff between a wide margin and few mistakes
n_support_	How many training points actually define the boundary
Same fit/predict calls	The consistent interface is why scikit-learn is popular

Step 5 — Evaluate properly

Accuracy alone can mislead. `classification_report` also gives precision (of the ones I said were species A, how many really were?) and recall (of all the real species A, how many did I find?). F1-score balances the two.

```
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix

print('=== Naïve Bayes ===')
print('Accuracy:', accuracy_score(y_test, nb_predictions))
print(classification_report(y_test, nb_predictions,
                           target_names=data.target_names))

print('=== SVM ===')
print('Accuracy:', accuracy_score(y_test, svm_predictions))
print(classification_report(y_test, svm_predictions,
                           target_names=data.target_names))

print('SVM confusion matrix:')
print(confusion_matrix(y_test, svm_predictions))
```

Line by line:

Code	What it does
accuracy_score	The fraction of predictions that were correct
precision	When the model says class A, how often is it right?
recall	Of all the real class A examples, how many did it catch?
f1-score	The balance between precision and recall
confusion_matrix	A grid showing exactly which classes got mixed up

The complete program (lab12.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab12.py – Comparing Naïve Bayes and SVM with scikit-learn
import time
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# --- 1. Load the data ---
data = load_iris()
X, y = data.data, data.target

print('Dataset:', X.shape[0], 'flowers,', X.shape[1], 'measurements each')
print('Species:', list(data.target_names))
print()

# --- 2. Split into train and test ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print('Training on', len(X_train), 'flowers; testing on', len(X_test))
print()

# --- 3. Train Naïve Bayes ---
nb_model = GaussianNB()
start = time.perf_counter()
nb_model.fit(X_train, y_train)
nb_time = time.perf_counter() - start
nb_predictions = nb_model.predict(X_test)

# --- 4. Train SVM ---
svm_model = SVC(kernel='rbf', C=1.0)
start = time.perf_counter()
svm_model.fit(X_train, y_train)
svm_time = time.perf_counter() - start
svm_predictions = svm_model.predict(X_test)

# --- 5. Evaluate both ---
print('=' * 55)
print('NAÏVE BAYES')
print('=' * 55)
print('Accuracy:      ', round(accuracy_score(y_test, nb_predictions), 4))
print('Training time:', round(nb_time, 5), 'seconds')
print()
print(classification_report(y_test, nb_predictions,
                           target_names=data.target_names))

print('=' * 55)
print('SUPPORT VECTOR MACHINE')
print('=' * 55)
print('Accuracy:      ', round(accuracy_score(y_test, svm_predictions), 4))
print('Training time:', round(svm_time, 5), 'seconds')
print('Support vectors per class:', svm_model.n_support_)
print()
print(classification_report(y_test, svm_predictions,
                           target_names=data.target_names))
```

```

print('=' * 55)
print('SIDE BY SIDE')
print('=' * 55)
print(f"{'Model':<16}{'Accuracy':<12}{'Train time (s)':<16}")
print(f"{'Naïve Bayes':<16}{accuracy_score(y_test, nb_predictions):<12.4f}{nb_time:<16.5f}")
print(f"{'SVM':<16}{accuracy_score(y_test, svm_predictions):<12.4f}{svm_time:<16.5f}")

```

HOW TO RUN IT

```
python lab12.py
```

WHAT YOU SHOULD SEE

Dataset: 150 flowers, 4 measurements each
 Species: ['setosa', 'versicolor', 'virginica']

Training on 120 flowers; testing on 30

NAÏVE BAYES

Accuracy: 0.9667
 Training time: 0.00089 seconds

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.91	1.00	0.95	10
virginica	1.00	0.90	0.95	10
accuracy			0.97	30

SUPPORT VECTOR MACHINE

Accuracy: 0.9667
 Training time: 0.00147 seconds
 Support vectors per class: [6 20 20]

SIDE BY SIDE

Model	Accuracy	Train time (s)
Naïve Bayes	0.9667	0.00089
SVM	0.9667	0.00147

(Both score identically here because Iris is an easy dataset. Notice setosa is always perfect – it is clearly separated – while versicolor and virginica overlap, which is where the single error occurs.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
ModuleNotFoundError: No module named 'sklearn'	The package name for installing differs from the import name	Install with: pip install scikit-learn (not 'sklearn')
Accuracy is suspiciously 1.0000	You tested on the training data instead of the test data	Always predict on X_test, never on X_train
Results differ from the handbook	random_state was not set, so the split differs each run	Set random_state=42 in train_test_split
ValueError: Unknown label type	y contains text labels where numbers were expected	Encode labels with LabelEncoder, or use the numeric data.target

TRY IT YOURSELF

- Change kernel to 'linear' and see whether accuracy changes. Iris is nearly linearly separable.
- Try $C=100$ and $C=0.01$. Very high C makes the SVM avoid all training mistakes, which can overfit.
- Swap in `load_wine()` or `load_breast_cancer()` and rerun the whole comparison.
- Write the comparison report your lab requires: which model won, by how much, and why their different assumptions explain the result.

LAB 13: Build and Train a Feedforward Neural Network for Digit Classification

Relates to: Weeks 13–14 — Neural Networks, MLP

WHAT THIS LAB IS ABOUT

MNIST is 70,000 photographs of handwritten digits, each 28 by 28 pixels. The task is to look at the pixels and say which digit it is. A neural network does this by learning weights: numbers that decide how strongly each pixel influences each possible answer. You will not calculate those weights yourself — TensorFlow does it. Your job is to describe the shape of the network and let it learn. This lab is the payoff for all the theory: roughly fifteen lines of code will produce a model that reads handwriting with about 98% accuracy.

BEFORE YOU START, YOU NEED

- Completion of Labs 1 and 12
- TensorFlow installed: pip install tensorflow
- An internet connection the first time, to download MNIST (about 11 MB)
- Patience — training takes one to three minutes on a normal laptop

Building it step by step

Step 1 — Load the images

TensorFlow downloads MNIST for you and hands back four arrays, already split into training and testing. The first time you run this it downloads; afterwards it is cached on your machine.

```
import tensorflow as tf

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print('Training images:', x_train.shape)    # (60000, 28, 28)
print('Training labels:', y_train.shape)    # (60000,)
print('Testing images: ', x_test.shape)     # (10000, 28, 28)
print()
print('First label:', y_train[0])
print('Pixel value range:', x_train.min(), 'to', x_train.max())
```

Line by line:

Code	What it does
(60000, 28, 28)	60,000 images, each 28 pixels tall and 28 wide
y_train	The correct digit for each image, from 0 to 9
0 to 255	Pixel brightness: 0 is black, 255 is white

Step 2 — See what an image actually looks like

Before trusting a model, always look at your data. This prints one digit as text art so you can confirm the images are what you think they are.

```
def show_digit(image, label):
    print('This should be a', label)
    for row in image:
        line = ''
        for pixel in row:
            if pixel > 200:
                line += '#'
            elif pixel > 100:
                line += '+'
```

```

        elif pixel > 0:
            line += '.'
        else:
            line += ' '
    print(line)

```

```
show_digit(x_train[0], y_train[0])
```

Line by line:

Code	What it does
for row in image	Each image is 28 rows of 28 pixel values
pixel > 200	Very bright, so draw a solid character
Text art	A quick sanity check requiring no plotting library

Step 3 — Prepare the data

Two changes are needed. First, flatten each 28x28 grid into one long row of 784 numbers, because a dense layer expects a flat list. Second, divide by 255 so every value sits between 0 and 1 — neural networks train far more reliably on small numbers.

```

# Flatten: (60000, 28, 28) becomes (60000, 784)
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

# Normalise: 0-255 becomes 0.0-1.0
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

print('New shape:', x_train.shape)
print('New range:', x_train.min(), 'to', x_train.max())

```

Line by line:

Code	What it does
reshape(-1, 784)	The -1 means 'work out this dimension for me'
784	28 multiplied by 28
astype('float32')	Converts whole numbers to decimals
/ 255.0	Normalisation — the same idea as in Lab 1

Step 4 — Describe the network

Sequential means the layers run one after another. Each Dense layer is fully connected to the one before it. The final layer has exactly 10 units, one per digit, and softmax turns their outputs into probabilities that add up to 1.

```

model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(784,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.summary()

```

Line by line:

Code	What it does
Dense(128)	A hidden layer of 128 neurons
activation='relu'	ReLU: keeps positives, turns negatives to zero. The standard choice
input_shape=(784,)	Tells the first layer how many inputs to expect
Dense(10)	Ten outputs, one per possible digit
softmax	Converts raw scores into probabilities summing to 1
model.summary()	Prints the layers and how many weights will be learned

Step 5 — Configure how it learns

Three choices. The optimizer decides how weights get adjusted — adam is a reliable default. The loss measures how wrong the model is; this particular one suits whole-number labels. The metric is what gets reported to you while training.

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Line by line:

Code	What it does
optimizer='adam'	The algorithm that performs gradient descent for you
sparse_categorical_crossentropy	Correct loss when labels are integers 0-9 rather than one-hot vectors
metrics=['accuracy']	What to display during training

Step 6 — Train, then evaluate

One epoch means one full pass through all 60,000 training images. `validation_split` holds back 10% of training data to check progress on unseen examples as it goes. Watch for validation accuracy plateauing while training accuracy keeps climbing — that is overfitting.

```
history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)

test_loss, test_accuracy = model.evaluate(x_test, y_test)
print()
print('Accuracy on unseen test images:', round(test_accuracy, 4))
```

Line by line:

Code	What it does
epochs=10	Ten complete passes through the training data
batch_size=32	Update the weights after every 32 images, not after all 60,000
validation_split=0.1	Reserve 10% to monitor progress honestly
model.evaluate	Final score on data never used in training at all

Step 7 — Inspect individual predictions

A single accuracy number hides what is really happening. Looking at specific mistakes tells you far more, and 4-versus-9 confusions in MNIST are famously common.

```
import numpy as np

predictions = model.predict(x_test)

# Find the first mistake
for i in range(len(x_test)):
    predicted = np.argmax(predictions[i])
    actual = y_test[i]
    if predicted != actual:
        print(f'Image {i}: predicted {predicted}, actually {actual}')
        confidence = predictions[i][predicted] * 100
        print(f'Model was {confidence:.1f}% confident')
        show_digit(x_test[i].reshape(28, 28) * 255, actual)
        break
```

Line by line:

Code	What it does
<code>np.argmax</code>	The position of the largest value — the predicted digit
<code>predictions[i][predicted]</code>	How confident the model was in that answer
<code>reshape(28, 28)</code>	Unflatten so we can display it again
<code>* 255</code>	Undo the normalisation, so <code>show_digit</code> 's thresholds work

The complete program (lab13.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab13.py – Handwritten digit recognition with a neural network
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

def show_digit(image, label):
    print('Label:', label)
    for row in image:
        line = ''
        for pixel in row:
            if pixel > 200: line += '#'
            elif pixel > 100: line += '+'
            elif pixel > 0: line += '.'
            else: line += ' '
        print(line)

# --- 1. Load ---
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
print('Training images:', x_train.shape)
print()
show_digit(x_train[0], y_train[0])
print()

# --- 2. Prepare ---
x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
x_test = x_test.reshape(-1, 784).astype('float32') / 255.0

# --- 3. Build ---
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(784,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.summary()

# --- 4. Configure ---
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# --- 5. Train ---
history = model.fit(x_train, y_train,
                    epochs=10, batch_size=32, validation_split=0.1)

# --- 6. Evaluate ---
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print()
print('Final test accuracy:', round(test_accuracy, 4))

# --- 7. Look at a mistake ---
predictions = model.predict(x_test)
for i in range(len(x_test)):
    predicted = np.argmax(predictions[i])
    if predicted != y_test[i]:
        print()
        print(f'MISTAKE on image {i}:')
```

```

        print(f' Predicted {predicted}, actual answer {y_test[i]}')
        print(f' Confidence: {predictions[i][predicted]*100:.1f}%')
        show_digit(x_test[i].reshape(28,28) * 255, y_test[i])
        break

# --- 8. Plot the learning curve ---
plt.plot(history.history['accuracy'], label='training')
plt.plot(history.history['val_accuracy'], label='validation')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('MNIST Training Progress')
plt.legend()
plt.savefig('lab13_curve.png')
print()
print('Learning curve saved as lab13_curve.png')

```

HOW TO RUN IT

```

pip install tensorflow          # first time only
python lab13.py

```

WHAT YOU SHOULD SEE

Training images: (60000, 28, 28)

Label: 5

```

...#####+.
.#####+.
+#####+....
#####+
+####.
+#####+
.#####+
..#####+
#####+
#####+

```

(etc.)

Model: "sequential"

```

Dense (None, 128)      100480 params
Dense (None, 64)       8256 params
Dense (None, 10)       650 params
Total params: 109,386

```

Epoch 1/10

1688/1688 [====] - 3s - accuracy: 0.9210 - val_accuracy: 0.9618

Epoch 5/10

1688/1688 [====] - 3s - accuracy: 0.9865 - val_accuracy: 0.9743

Epoch 10/10

1688/1688 [====] - 3s - accuracy: 0.9938 - val_accuracy: 0.9768

313/313 [====] - accuracy: 0.9772

Final test accuracy: 0.9772

MISTAKE on image 8:

```

Predicted 6, actual answer 5
Confidence: 62.3%

```

Learning curve saved as lab13_curve.png

(Roughly 97-98% accuracy is normal for this architecture. Notice training accuracy reaches 99.4% while validation stalls near 97.7% — the gap is mild overfitting, exactly as covered in Week 11.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
ModuleNotFoundError: No module named 'tensorflow'	TensorFlow is not installed	Run: pip install tensorflow (it is a large download, roughly 500 MB)
Download fails on first run	No internet connection, or a firewall blocked it	Connect to the internet; MNIST is cached after the first successful download
ValueError: Input 0 of layer is incompatible	You forgot to flatten the images to 784 values	Apply reshape(-1, 784) before training
Accuracy stuck around 10%	The data was not normalised, so gradients exploded	Divide pixel values by 255.0
Training is extremely slow	This is normal on a laptop without a GPU	Reduce epochs to 3 while experimenting

TRY IT YOURSELF

- Remove the middle Dense(64) layer. Does accuracy drop much?
- Change epochs to 30 and watch the gap between training and validation accuracy widen — textbook overfitting.
- Add tf.keras.layers.Dropout(0.2) between the dense layers and see whether the gap narrows.
- Replace relu with sigmoid and observe how much more slowly it learns — this is the vanishing gradient problem from Week 14.

LAB 14: Implement an MLP From Scratch to Solve XOR

Relates to: Weeks 13–14 — Perceptron limits, Backpropagation

WHAT THIS LAB IS ABOUT

In Lab 13 TensorFlow did all the calculus for you. Now you do it yourself. XOR outputs 1 when exactly one input is 1. It is famous because a single-layer perceptron provably cannot learn it — you cannot draw one straight line separating the two 1s from the two 0s. Adding a hidden layer fixes this, but only if you can train it, and training a hidden layer requires backpropagation: working out how much each weight contributed to the error by applying the chain rule backwards through the network. Every line of that is written out here, using nothing but NumPy.

BEFORE YOU START, YOU NEED

- Completion of Labs 1 and 13
- numpy installed
- Awareness of what a derivative is (you do not need to calculate any)

Building it step by step

Step 1 — See why one layer cannot work

Plot the four XOR points on paper: (0,0) and (1,1) output 0; (0,1) and (1,0) output 1. The two output classes sit on opposite diagonal corners. No single straight line can separate them. This is not a training problem, it is a structural impossibility.

```
#  input1  input2  |  XOR output
#  -----  -----  |  -----
#    0      0      |    0
#    0      1      |    1
#    1      0      |    1
#    1      1      |    0

# Plotted:
#  1 | 1      0      <- (0,1)=1   (1,1)=0
#   |
#  0 | 0      1      <- (0,0)=0   (1,0)=1
#   +-----
#     0      1

# The two 1s are diagonally opposite. No straight line separates
# them from the two 0s. Hence a single-layer perceptron fails.
```

Step 2 — Set up the data and the activation function

Sigmoid squashes any number into the range 0 to 1, giving the network its non-linearity. Its derivative has a beautifully simple form when you already know the output: output times (1 minus output). That simplicity is why sigmoid is used for teaching backpropagation.

```
import numpy as np

np.random.seed(1)          # reproducible results

X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

y = np.array([0],
              [1],
              [1],
              [0])
```

```

[0]])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(output):
    # Note: takes the OUTPUT of sigmoid, not the input
    return output * (1 - output)

print('sigmoid(0)  =', sigmoid(0))      # 0.5
print('sigmoid(5)  =', sigmoid(5))      # 0.993
print('sigmoid(-5) =', sigmoid(-5))     # 0.007

```

Line by line:

Code	What it does
X shape (4, 2)	Four examples, two inputs each
y shape (4, 1)	Four examples, one output each
sigmoid	Squashes anything into the range 0 to 1
output * (1 - output)	The derivative, expressed using the output we already have

Step 3 — Create the weights

The network has 2 inputs, 4 hidden neurons, and 1 output. W1 connects inputs to hidden, so it must be 2 by 4. W2 connects hidden to output, so it is 4 by 1. Weights start random — if they all started identical, every neuron would learn exactly the same thing.

```

input_size = 2
hidden_size = 4
output_size = 1
learning_rate = 0.5

W1 = np.random.uniform(-1, 1, (input_size, hidden_size)) # 2 x 4
b1 = np.zeros((1, hidden_size))                          # 1 x 4

W2 = np.random.uniform(-1, 1, (hidden_size, output_size)) # 4 x 1
b2 = np.zeros((1, output_size))                          # 1 x 1

print('W1 shape:', W1.shape)
print('W2 shape:', W2.shape)

```

Line by line:

Code	What it does
np.random.uniform(-1, 1, shape)	Random numbers between -1 and 1
(2, 4)	Two inputs connecting to four hidden neurons
b1, b2	Biases — an extra adjustable number per neuron, starting at zero
learning_rate	How big a step to take when adjusting weights

Step 4 — The forward pass

Multiply the inputs by the weights, add the bias, squash with sigmoid. Do that twice: once to get the hidden layer, once to get the output. np.dot does the matrix multiplication for all four examples at once.

```

# Layer 1: inputs -> hidden
hidden_input = np.dot(X, W1) + b1

```

```

hidden_output = sigmoid(hidden_input)

# Layer 2: hidden -> output
final_input = np.dot(hidden_output, W2) + b2
final_output = sigmoid(final_input)

print('Hidden layer shape:', hidden_output.shape) # (4, 4)
print('Output shape:      ', final_output.shape)  # (4, 1)
print()
print('Untrained predictions:')
print(final_output)
print('(All near 0.5 – the network knows nothing yet.)')

```

Line by line:

Code	What it does
<code>np.dot(X, W1)</code>	Matrix multiplication: processes all 4 examples simultaneously
<code>+ b1</code>	Broadcasting: the bias row is added to every example
<code>sigmoid(...)</code>	Applies the squashing function to every value
Near 0.5	Random weights give roughly random guesses

Step 5 — The backward pass (backpropagation)

This is the heart of the lab. Read each line with its comment. We find the error at the output, scale it by how sensitive the output was (its derivative), then push that error backwards through $W2$ to discover how much the hidden layer was to blame, and scale again by the hidden layer's derivative.

```

# 1. How wrong were we?
error = y - final_output

# 2. How much should the output change?
# Scale the error by how sensitive the output was
d_output = error * sigmoid_derivative(final_output)

# 3. Push the blame backwards through W2 to the hidden layer
error_hidden = d_output.dot(W2.T)

# 4. Scale by the hidden layer's own sensitivity
d_hidden = error_hidden * sigmoid_derivative(hidden_output)

# 5. Update the weights, each scaled by its own input
W2 += hidden_output.T.dot(d_output) * learning_rate
b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate

W1 += X.T.dot(d_hidden) * learning_rate
b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

```

Line by line:

Code	What it does
<code>y - final_output</code>	The raw error: target minus prediction
<code>* sigmoid_derivative(...)</code>	The chain rule: scale by the local gradient
<code>d_output.dot(W2.T)</code>	THE key backpropagation step — error flows backwards through the weights
<code>W2.T</code>	Transpose: flips rows and columns so the shapes line up
<code>hidden_output.T.dot(d_output)</code>	Gradient for $W2$: how much each weight should change
<code>+=</code>	Gradient ascent on the error we want to reduce, hence plus not minus here

Step 6 — Put it in a training loop

Repeat the forward and backward passes ten thousand times. Each repetition nudges the weights slightly closer to something that works. Print the loss occasionally so you can watch it fall.

```
for epoch in range(10000):
    # Forward
    hidden_output = sigmoid(np.dot(X, W1) + b1)
    final_output = sigmoid(np.dot(hidden_output, W2) + b2)

    # Backward
    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)
    d_hidden = d_output.dot(W2.T) * sigmoid_derivative(hidden_output)

    # Update
    W2 += hidden_output.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    W1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

    if epoch % 2000 == 0:
        loss = np.mean(np.square(error))
        print(f'Epoch {epoch:5d}   Loss: {loss:.6f}')
```

Line by line:

Code	What it does
for epoch in range(10000)	Ten thousand rounds of learning
epoch % 2000 == 0	Print only every 2000th round, to keep output readable
np.mean(np.square(error))	Mean squared error — the standard loss measure
Loss falling	Proof that learning is happening

The complete program (lab14.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab14.py – An MLP built from scratch, solving XOR via backpropagation
import numpy as np

np.random.seed(1)

# --- The XOR problem ---
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([[0], [1], [1], [0]])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(output):
    return output * (1 - output)

# --- Network shape ---
input_size, hidden_size, output_size = 2, 4, 1
learning_rate = 0.5

W1 = np.random.uniform(-1, 1, (input_size, hidden_size))
b1 = np.zeros((1, hidden_size))
W2 = np.random.uniform(-1, 1, (hidden_size, output_size))
b2 = np.zeros((1, output_size))

print('Network: 2 inputs -> 4 hidden -> 1 output')
print('Training for 10,000 epochs...')
print()

# --- Training loop ---
for epoch in range(10000):

    # FORWARD PASS
    hidden_output = sigmoid(np.dot(X, W1) + b1)
    final_output = sigmoid(np.dot(hidden_output, W2) + b2)

    # BACKWARD PASS
    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)
    d_hidden = d_output.dot(W2.T) * sigmoid_derivative(hidden_output)

    # WEIGHT UPDATES
    W2 += hidden_output.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    W1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

    if epoch % 2000 == 0:
        loss = np.mean(np.square(error))
        print(f'Epoch {epoch:5d} Loss: {loss:.6f}')

# --- Results ---
print()
print('=== Final predictions ===')
print(f'{"Input":<10}{"Predicted":<14}{"Target":<10}{"Correct?":<10}')
print('-' * 45)
```

```

for i in range(4):
    predicted = final_output[i][0]
    target = y[i][0]
    rounded = round(predicted)
    mark = 'yes' if rounded == target else 'NO'
    print(f'{str(X[i]):<10}{predicted:<14.4f}{target:<10}{mark}')

print()
print('A single-layer perceptron can NEVER do this.')
print('The hidden layer is what makes it possible.')

```

HOW TO RUN IT

```
python lab14.py
```

WHAT YOU SHOULD SEE

```

Network: 2 inputs -> 4 hidden -> 1 output
Training for 10,000 epochs...

```

```

Epoch    0    Loss: 0.283746
Epoch 2000    Loss: 0.002145
Epoch 4000    Loss: 0.000834
Epoch 6000    Loss: 0.000489
Epoch 8000    Loss: 0.000338

```

```
=== Final predictions ===
```

Input	Predicted	Target	Correct?
[0 0]	0.0169	0	yes
[0 1]	0.9847	1	yes
[1 0]	0.9845	1	yes
[1 1]	0.0170	0	yes

```

A single-layer perceptron can NEVER do this.
The hidden layer is what makes it possible.

```

```

(Predictions near 0.02 and 0.98 instead of exactly 0 and 1 are expected –
sigmoid never quite reaches its limits. Rounding gives perfect answers.)

```

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
Loss stops falling around 0.25	The network got stuck; often caused by unlucky initial weights	Change np.random.seed, or raise hidden_size to 8
ValueError: shapes not aligned	A matrix transpose is missing	Check W2.T and X.T are present; print .shape on each array to diagnose
All four outputs are 0.5	The weight updates are not being applied	Confirm the += lines are inside the training loop
RuntimeWarning: overflow in exp	Learning rate too high, so weights exploded	Reduce learning_rate to 0.1
Predictions are inverted	y was entered in the wrong order	XOR targets are 0, 1, 1, 0 for inputs 00, 01, 10, 11

TRY IT YOURSELF

- Set hidden_size to 1 and try to train. It will fail — one hidden neuron is not enough to bend the decision boundary.
- Change the targets to AND (0,0,0,1). It learns almost instantly, because AND IS linearly separable.
- Try learning_rate of 0.01 and of 5.0. One is painfully slow, the other unstable.
- Print the hidden layer outputs after training and try to work out what each neuron learned to detect.

LAB 15: Implement a Genetic Algorithm to Solve Eight Queens

Relates to: Week 15 — Genetic Algorithms

WHAT THIS LAB IS ABOUT

Hill climbing in Lab 6 kept one solution and got stuck 86% of the time. A genetic algorithm keeps a whole population of solutions and breeds them. Good solutions get more chances to reproduce. Reproduction means crossover: take the first part of one parent and the second part of another. Occasionally a random mutation introduces something neither parent had. Over generations, the population improves. Nothing here understands chess — the algorithm only knows which boards score better, and lets selection do the rest.

BEFORE YOU START, YOU NEED

- Completion of Lab 6 (same problem, same board representation)
- Understanding of Python lists and the random module

Building it step by step

Step 1 — Encode a solution as a chromosome

We reuse Lab 6's representation exactly: eight numbers, one per column, giving that queen's row. In genetic algorithm language this list is called a chromosome, and each number is a gene.

```
import random

N = 8

def random_individual():
    return [random.randint(0, N-1) for _ in range(N)]

example = random_individual()
print('A chromosome:', example)
print('Gene 0 means the queen in column 0 sits in row', example[0])
```

Line by line:

Code	What it does
chromosome	One candidate solution, encoded as a list
gene	One number in that list
random.randint(0, N-1)	A random row from 0 to 7

Step 2 — Define fitness (higher is better)

Genetic algorithms maximise fitness, but 8-Queens is about minimising conflicts. We flip it: there are 28 possible pairs of queens in total, so fitness equals 28 minus the number of attacking pairs. A perfect board scores 28.

```
MAX_FITNESS = 28      # C(8,2) = 8*7/2 = 28 possible pairs

def fitness(individual):
    attacking = 0
    for i in range(N):
        for j in range(i+1, N):
            same_row = individual[i] == individual[j]
            same_diag = abs(individual[i]-individual[j]) == abs(i-j)
            if same_row or same_diag:
                attacking += 1
    return MAX_FITNESS - attacking
```

```
print('Fitness of example:', fitness(example))
print('A perfect solution would score:', MAX_FITNESS)
```

Line by line:

Code	What it does
MAX_FITNESS - attacking	Flips minimisation into maximisation
Fitness 28	Zero conflicts — a genuine solution
Same conflict logic	Identical to attacking_pairs in Lab 6

Step 3 — Selection: choose who reproduces

Tournament selection is simple and effective. Pick five individuals at random, and let the fittest of those five become a parent. Fitter individuals win tournaments more often, but weaker ones occasionally slip through, which preserves diversity.

```
def tournament_selection(population, k=5):
    contenders = random.sample(population, k)
    return max(contenders, key=fitness)

# Demonstration
population = [random_individual() for _ in range(20)]
parent = tournament_selection(population)
print('Tournament winner has fitness', fitness(parent))
```

Line by line:

Code	What it does
random.sample(population, k)	Picks k distinct individuals at random
max(..., key=fitness)	The fittest of those k
k=5	Larger k means stronger pressure towards the very best

Step 4 — Crossover: combine two parents

Pick a random cut point. The child takes everything before the cut from parent one, and everything after from parent two. This is how good partial solutions from different individuals get combined.

```
def crossover(parent1, parent2):
    point = random.randint(1, N-1)
    child = parent1[:point] + parent2[point:]
    return child

p1 = [0,1,2,3,4,5,6,7]
p2 = [7,6,5,4,3,2,1,0]

random.seed(3)
child = crossover(p1, p2)
print('Parent 1:', p1)
print('Parent 2:', p2)
print('Child:    ', child)
```

Line by line:

Code	What it does
point	Where to cut, somewhere between 1 and 7
parent1[:point]	Everything up to but not including the cut
parent2[point:]	Everything from the cut onwards

Code	What it does
+	Joins the two pieces into a new chromosome

Step 5 — Mutation: introduce novelty

With a small probability, move one queen to a random row. Without mutation the population eventually becomes identical and stops improving. Too much mutation and the search becomes random guessing. Ten percent is a reasonable starting point.

```
MUTATION_RATE = 0.1

def mutate(individual):
    if random.random() < MUTATION_RATE:
        column = random.randint(0, N-1)
        individual[column] = random.randint(0, N-1)
    return individual

before = [0,1,2,3,4,5,6,7]
random.seed(2)
after = mutate(list(before))
print('Before:', before)
print('After: ', after)
```

Line by line:

Code	What it does
random.random()	A decimal between 0 and 1
< MUTATION_RATE	True roughly 10% of the time
list(before)	Copies, so the original stays unchanged
Purpose	Keeps genetic diversity alive so evolution can continue

Step 6 — The generational loop

Sort the population by fitness. If the best is perfect, stop. Otherwise keep the top few unchanged (elitism, so we never lose our best work), and fill the rest of the new generation with children of selected parents.

```
POPULATION_SIZE = 100
MAX_GENERATIONS = 1000
ELITE_COUNT = 5

def genetic_algorithm():
    population = [random_individual() for _ in range(POPULATION_SIZE)]

    for generation in range(MAX_GENERATIONS):
        population.sort(key=fitness, reverse=True)

        if fitness(population[0]) == MAX_FITNESS:
            return population[0], generation

        # Elitism: carry the very best forward untouched
        new_population = population[:ELITE_COUNT]

        while len(new_population) < POPULATION_SIZE:
            parent1 = tournament_selection(population)
            parent2 = tournament_selection(population)
            child = crossover(parent1, parent2)
```

```
        child = mutate(child)
        new_population.append(child)

    population = new_population

    return population[0], MAX_GENERATIONS
```

Line by line:

Code	What it does
<code>sort(key=fitness, reverse=True)</code>	Fittest individuals first
<code>population[:ELITE_COUNT]</code>	Elitism — the best 5 survive unchanged
<code>while len(...) < POPULATION_SIZE</code>	Keep breeding until the new generation is full
<code>return population[0], generation</code>	The solution, plus how many generations it took

The complete program (lab15.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab15.py – Solving Eight Queens with a genetic algorithm
import random

N = 8
MAX_FITNESS = 28
POPULATION_SIZE = 100
MUTATION_RATE = 0.1
MAX_GENERATIONS = 1000
ELITE_COUNT = 5

def random_individual():
    return [random.randint(0, N-1) for _ in range(N)]

def fitness(individual):
    attacking = 0
    for i in range(N):
        for j in range(i+1, N):
            if individual[i] == individual[j] or \
               abs(individual[i]-individual[j]) == abs(i-j):
                attacking += 1
    return MAX_FITNESS - attacking

def tournament_selection(population, k=5):
    return max(random.sample(population, k), key=fitness)

def crossover(parent1, parent2):
    point = random.randint(1, N-1)
    return parent1[:point] + parent2[point:]

def mutate(individual):
    if random.random() < MUTATION_RATE:
        column = random.randint(0, N-1)
        individual[column] = random.randint(0, N-1)
    return individual

def show_board(state):
    for row in range(N):
        line = ''
        for col in range(N):
            line += 'Q ' if state[col] == row else '. '
        print(line)

def genetic_algorithm(verbose=True):
    population = [random_individual() for _ in range(POPULATION_SIZE)]

    for generation in range(MAX_GENERATIONS):
        population.sort(key=fitness, reverse=True)
        best_fitness = fitness(population[0])

        if verbose and generation % 10 == 0:
            average = sum(fitness(i) for i in population) / len(population)
            print(f'Gen {generation:4d} best={best_fitness} avg={average:.2f}')

        if best_fitness == MAX_FITNESS:
            return population[0], generation
```

```

    new_population = population[:ELITE_COUNT]
    while len(new_population) < POPULATION_SIZE:
        parent1 = tournament_selection(population)
        parent2 = tournament_selection(population)
        child = mutate(crossover(parent1, parent2))
        new_population.append(child)
    population = new_population

    return population[0], MAX_GENERATIONS

random.seed(7)
print('Evolving a solution to Eight Queens...')
print()

solution, generations = genetic_algorithm()

print()
print('=== Result ===')
print('Chromosome:', solution)
print('Fitness:', fitness(solution), 'out of', MAX_FITNESS)
print('Generations needed:', generations)
print()
show_board(solution)
print()
print('Compare with Lab 6: plain hill climbing solved only ~14% of')
print('random starts. The population-based approach escapes local optima.')

```

HOW TO RUN IT

```
python lab15.py
```

WHAT YOU SHOULD SEE

Evolving a solution to Eight Queens...

```

Gen    0  best=26  avg=21.84
Gen   10  best=27  avg=25.61
Gen   20  best=27  avg=26.12
Gen   30  best=27  avg=26.44

```

```

=== Result ===
Chromosome: [3, 1, 6, 2, 5, 7, 4, 0]
Fitness: 28 out of 28
Generations needed: 37

```

```

. . . . . Q
. Q . . . .
. . . Q . .
Q . . . . .
. . . . . Q .
. . . . Q . .
. . Q . . . .
. . . . . Q .

```

Compare with Lab 6: plain hill climbing solved only ~14% of random starts. The population-based approach escapes local optima.

(Watch the avg column climb steadily – that is the whole population improving, not just one lucky individual. The exact generation count varies between runs because the algorithm is stochastic.)

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
ValueError: Sample larger than population	Tournament size k exceeds the population size	Ensure POPULATION_SIZE is comfortably larger than k
Fitness never improves	Elitism is missing, so good solutions get lost each generation	Carry the top few individuals forward unchanged
Solution found instantly every time	MAX_FITNESS is wrong, so any board appears perfect	For $N=8$ the value must be 28
Population becomes all identical	Mutation rate is too low	Raise MUTATION_RATE to 0.2 and observe the difference
Never converges within 1000 generations	Mutation rate too high, turning the search random	Lower MUTATION_RATE to 0.05

TRY IT YOURSELF

- Set MUTATION_RATE to 0 and watch the population stagnate — evolution needs novelty.
- Set MUTATION_RATE to 0.9 and watch it behave like random search.
- Reduce POPULATION_SIZE to 10 and see how many more generations are needed.
- Change N to 16 and solve Sixteen Queens — remember to update MAX_FITNESS to $N*(N-1)/2$.

LAB 16: Build a Small AI Application Using a Pre-Trained CNN

Relates to: Week 16 — Deep Learning, CNNs

WHAT THIS LAB IS ABOUT

Training an image recognition network from scratch requires millions of pictures and days of computation. Nobody does that. Instead we use transfer learning: download a network that a large organisation already trained on millions of images, and reuse it. The early layers of such a network have learned universal visual features — edges, textures, shapes — that are useful for almost any image task. You either use the network as-is, or replace just its final layer and retrain that on your own small dataset. This capstone lab does both.

BEFORE YOU START, YOU NEED

- Completion of Lab 13
- TensorFlow installed
- Internet connection for the first run (downloads about 14 MB of weights)
- A test image saved in your folder as `sample_image.jpg`

Building it step by step

Step 1 — Download a pre-trained network

MobileNetV2 was trained on ImageNet: 1.2 million photographs across 1000 categories. Downloading it gives you all that learning for free. It is deliberately designed to be small and fast enough for phones.

```
import tensorflow as tf

model = tf.keras.applications.MobileNetV2(weights='imagenet')

print('Model loaded.')
print('Expected input shape:', model.input_shape)  # (None, 224, 224, 3)
print('Number of output classes:', model.output_shape[1])  # 1000
```

Line by line:

Code	What it does
<code>weights='imagenet'</code>	Download the already-trained weights
<code>(None, 224, 224, 3)</code>	Any number of images, each 224x224 pixels, 3 colour channels
<code>1000</code>	The number of categories it was trained to recognise
<code>None</code>	The batch dimension — it accepts any number of images at once

Step 2 — Prepare an image correctly

The model expects images in a very specific format. Get this wrong and you will get nonsense predictions rather than an error. Three things must happen: resize to 224x224, add a batch dimension, and apply the exact preprocessing this model was trained with.

```
import numpy as np

def load_and_preprocess(image_path):
    # 1. Load and resize to exactly what the model expects
    img = tf.keras.utils.load_img(image_path, target_size=(224, 224))

    # 2. Convert to numbers: shape becomes (224, 224, 3)
    img_array = tf.keras.utils.img_to_array(img)

    # 3. Add a batch dimension: (1, 224, 224, 3)
```

```
img_array = np.expand_dims(img_array, axis=0)

# 4. Apply MobileNetV2's own preprocessing
return tf.keras.applications.mobilenet_v2.preprocess_input(img_array)
```

Line by line:

Code	What it does
target_size=(224, 224)	Resize — non-negotiable, the model requires this
img_to_array	Turns the picture into a grid of numbers
np.expand_dims(..., axis=0)	Adds the batch dimension the model expects
preprocess_input	Scales pixels the specific way MobileNetV2 was trained on
Skipping step 4	A common bug: gives confident but completely wrong answers

Step 3 — Make a prediction

The model returns 1000 numbers, one probability per category. `decode_predictions` does the hard work of matching those numbers to readable names and picking the highest.

```
def predict_image(image_path, top_k=3):
    processed = load_and_preprocess(image_path)

    predictions = model.predict(processed, verbose=0)

    decoded = tf.keras.applications.mobilenet_v2.decode_predictions(
        predictions, top=top_k
    )[0]

    return [(label, float(confidence)) for (_, label, confidence) in decoded]

results = predict_image('sample_image.jpg')

print('Top predictions:')
for label, confidence in results:
    print(f' {label:<25} {confidence*100:5.2f}%')
```

Line by line:

Code	What it does
model.predict(...)	Runs the image through the network
decode_predictions	Converts 1000 raw numbers into readable category names
[0]	We passed one image, so take the first (only) result
(_, label, confidence)	Each result has an id, a name and a score; we ignore the id

Step 4 — Build a custom classifier by transfer learning

Now the real technique. `include_top=False` removes the original 1000-category output layer, leaving just the feature extractor. Setting `trainable=False` freezes it so all that learned knowledge is preserved. Then we bolt on our own small output layer for our own categories.

```
base_model = tf.keras.applications.MobileNetV2(
    weights='imagenet',
    include_top=False,           # drop the original output layer
    input_shape=(224, 224, 3)
)
```

```

base_model.trainable = False          # freeze all the learned features

custom_model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')    # our 3 categories
])

custom_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

custom_model.summary()

```

Line by line:

Code	What it does
<code>include_top=False</code>	Keep the feature extractor, discard the classifier
<code>trainable = False</code>	Freeze — do not undo millions of images worth of learning
<code>GlobalAveragePooling2D</code>	Flattens the feature maps into a single row of numbers
<code>Dense(3)</code>	Change this to your own number of categories
<code>Only the new layers train</code>	Which is why this works with only a few hundred images

Step 5 — Train on your own images (optional extension)

Organise your images into one folder per category. `image_dataset_from_directory` reads them and works out the labels from the folder names automatically. Because the base is frozen, a handful of epochs is usually enough.

```

# Expected folder layout:
#   my_images/
#     cats/      (photos of cats)
#     dogs/      (photos of dogs)
#     birds/     (photos of birds)

train_dataset = tf.keras.utils.image_dataset_from_directory(
    'my_images',
    image_size=(224, 224),
    batch_size=16,
    validation_split=0.2,
    subset='training',
    seed=42
)

val_dataset = tf.keras.utils.image_dataset_from_directory(
    'my_images',
    image_size=(224, 224),
    batch_size=16,
    validation_split=0.2,
    subset='validation',
    seed=42
)

custom_model.fit(train_dataset, validation_data=val_dataset, epochs=5)

```

Line by line:

Code	What it does
<code>image_dataset_from_directory</code>	Folder names automatically become the category labels
<code>validation_split=0.2</code>	Holds back a fifth for honest evaluation
<code>seed=42</code>	Ensures training and validation subsets do not overlap
<code>epochs=5</code>	Few epochs needed, since only the small new head is learning

The complete program (lab16.py)

Type or paste this into a file and run it. Every line above appears here in order.

```
# lab16.py – Image classification with a pre-trained CNN
import tensorflow as tf
import numpy as np
import os

print('Loading MobileNetV2 (first run downloads ~14 MB)...')
model = tf.keras.applications.MobileNetV2(weights='imagenet')
print('Loaded. Input shape:', model.input_shape)
print()

def load_and_preprocess(image_path):
    img = tf.keras.utils.load_img(image_path, target_size=(224, 224))
    img_array = tf.keras.utils.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    return tf.keras.applications.mobilenet_v2.preprocess_input(img_array)

def predict_image(image_path, top_k=3):
    processed = load_and_preprocess(image_path)
    predictions = model.predict(processed, verbose=0)
    decoded = tf.keras.applications.mobilenet_v2.decode_predictions(
        predictions, top=top_k)[0]
    return [(label, float(conf)) for (_, label, conf) in decoded]

# --- Part 1: classify an image using the model as-is ---
IMAGE = 'sample_image.jpg'

if os.path.exists(IMAGE):
    print('=' * 50)
    print('CLASSIFYING:', IMAGE)
    print('=' * 50)
    for label, confidence in predict_image(IMAGE):
        bar = '#' * int(confidence * 40)
        print(f' {label:<22} {confidence*100:5.2f}% {bar}')
else:
    print(f'No {IMAGE} found. Save any photo with that name to test.')

print()

# --- Part 2: build a custom classifier by transfer learning ---
print('=' * 50)
print('BUILDING A CUSTOM CLASSIFIER')
print('=' * 50)

base_model = tf.keras.applications.MobileNetV2(
    weights='imagenet', include_top=False, input_shape=(224,224,3))
base_model.trainable = False

NUM_CLASSES = 3

custom_model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(NUM_CLASSES, activation='softmax')
])
```

```

custom_model.compile(optimizer='adam',
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])

custom_model.summary()

trainable = sum(1 for layer in custom_model.layers if layer.trainable)
print()
print(f'Frozen base + {NUM_CLASSES}-class head ready for training.')
print('Organise images as my_images/class1, my_images/class2, ... then:')
print("  ds = tf.keras.utils.image_dataset_from_directory('my_images',"
        "        image_size=(224,224), batch_size=16)")
print('  custom_model.fit(ds, epochs=5)')

```

HOW TO RUN IT

```

pip install tensorflow pillow      # pillow is needed to read images
# save any photo in the folder as sample_image.jpg
python lab16.py

```

WHAT YOU SHOULD SEE

```

Loading MobileNetV2 (first run downloads ~14 MB)...
Loaded. Input shape: (None, 224, 224, 3)

=====
CLASSIFYING: sample_image.jpg
=====
golden_retriever      87.32% #####
Labrador_retriever    6.41% ##
kuvasz                 1.87%

=====
BUILDING A CUSTOM CLASSIFIER
=====
Model: "sequential"
mobilenetv2_1.00_224 (Functional) 2,257,984 params [frozen]
global_average_pooling2d          0 params
dense (64 units)                   81,984 params
dense_1 (3 units)                  195 params
Total params: 2,340,163
Trainable params: 82,179
Non-trainable params: 2,257,984

Frozen base + 3-class head ready for training.

(Note the numbers: 2.26 million frozen parameters carry all the visual
knowledge, while only 82,000 trainable parameters need to learn your
specific task. That ratio is exactly why transfer learning works with
small datasets.)

```

IF SOMETHING GOES WRONG

Error message / symptom	Why it happened	How to fix it
ModuleNotFoundError: No module named 'PIL'	Pillow, the image library, is not installed	Run: <code>pip install pillow</code>
FileNotFoundError: sample_image.jpg	No test image in the folder	Save any JPEG photo in the same folder with that exact name
Predictions are confident but absurd	preprocess_input was skipped	Always call <code>preprocess_input</code> before predicting
ValueError: Input 0 is incompatible	The image was not resized to 224x224, or	Check both <code>target_size</code> and <code>np.expand_dims</code>

Error message / symptom	Why it happened	How to fix it
with layer	the batch dimension is missing	
Download fails	No internet, or a proxy blocks the request	Connect to the internet; weights are cached after one successful download
Custom model learns nothing	The base was left trainable, so it is being destroyed by a tiny dataset	Set <code>base_model.trainable = False</code>

TRY IT YOURSELF

- Classify several different photos and note where it succeeds and where it fails. Try an unusual angle or partial object.
- Swap MobileNetV2 for ResNet50 and compare accuracy and speed.
- Collect 30 photos each of three categories you care about and actually train the custom head.
- After training the head, unfreeze the last few base layers and fine-tune with a very low learning rate (1e-5). This is full fine-tuning.
- Wrap `predict_image` in a simple loop that classifies every JPEG in a folder — a complete, useful mini application.